

Datos Generales del Problemario

Profesor: M. en C. Manuel Olán Ramos		Asignatura: Programación Avanzada
Semestre y grupo: 7-C	Fecha de entrega: 09/10/2026	Unidad: 2
Alumnos: De La Fuente Chapuz Enrique. Fuentes de la Cruz Bernardo. Hernández Domínguez Bryan.		

Problemario #2: Programación Orientada a Objetos.

Instrucciones:

Realiza los siguientes ejercicios como un proyecto independiente de Aplicación de consola en C# dentro de Visual Studio. Para cada ejercicio deberán entregar:

Parte 1: Desarrollo a mano en hojas blancas:

- 1) Identificar claramente las entradas, el proceso y las salidas del problema.
- 2) Redactar el pseudocódigo completo. Debe incluir las decisiones y ciclos que correspondan.
- 3) Dibujar el diagrama de flujo usando símbolos correctos: inicio y fin, entrada y salida, proceso, decisión y flechas de dirección.
- 4) Determinar y elaborar una tabla con: a) Las clases, b) Objetos creados a partir de las clases, c) Propiedades (especificar el tipo de dato) y d) Los métodos (especificar sus parámetros).

Parte 2: Desarrollo en Visual Studio:

- 1) Crear un proyecto independiente de tipo Aplicación de consola para cada ejercicio.
- 2) Implementar las clases, objetos, propiedades y métodos propuestos en la parte escrita.
- 3) Incluir comentarios breves para indicar las clases, objetos, propiedades y métodos.
- 4) Mostrar unidades en todas las salidas numéricas cuando corresponda, por ejemplo: V, A, W, N, °C, mm/s, etc.
- 5) Ejecutar las pruebas y anexar capturas legibles de la consola.
- 6) Comprobar que el programa no produzca divisiones entre cero ni ciclos infinitos.
- 7) Guardar sus códigos en su repositorio local y GitHub.

Ejercicio 1:

Realizado por: Fuentes de la Cruz Bernardo.

Una estación de prueba necesita evaluar el consumo eléctrico de un motor de corriente directa. El usuario introducirá el identificador del motor, el voltaje de alimentación y la corriente consumida. El programa calculará la potencia eléctrica mediante $P = V \cdot I$. Para este ejercicio, una potencia menor o igual que 120 W se considera consumo normal; una potencia mayor se considera consumo elevado.

Parte 1: Desarrollo a mano:

Parte 1 - Ejercicio 1.

1) Identificar los entornos, el proceso y las salidas del programa.

- Entradas: Identificador, Voltaje, Corriente
- Proceso: $P = V \cdot I$. Evaluar si $P \leq 120$.
- Salidas: Identificador, Voltaje, Corriente, Potencia, y el estado ("Consumo normal" o "Consumo elevado").

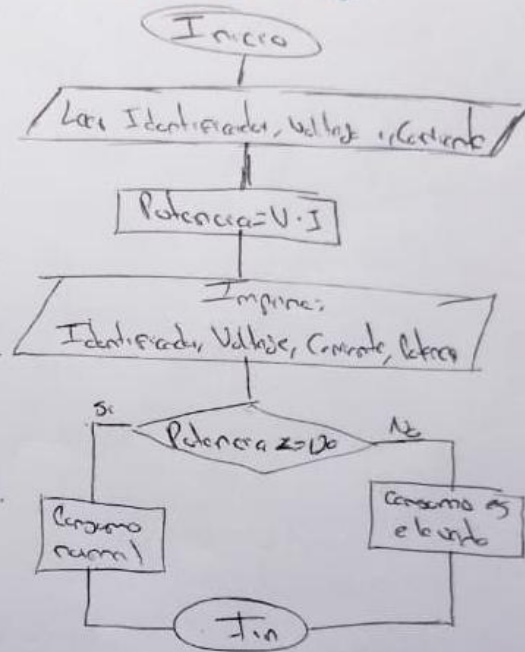
2) Redactar el Pseudocódigo completo. Debe incluir las decisiones y ciclos que corresponda.

Inicio

- Crear el objeto motor.
- Pedir y Leer Identificador, Voltaje y Corriente
- Llamar al motor
- Definición de Clase
- Propiedades: Identificador, Voltaje y Corriente
- Calcular Potencia: Voltaje * Corriente.
- Método de Consumo
 - si Potencia ≤ 120
Imprimir "Consumo normal"
 - Si no
Imprimir "Consumo elevado"

Fina

3) Diagrama de Flujo.

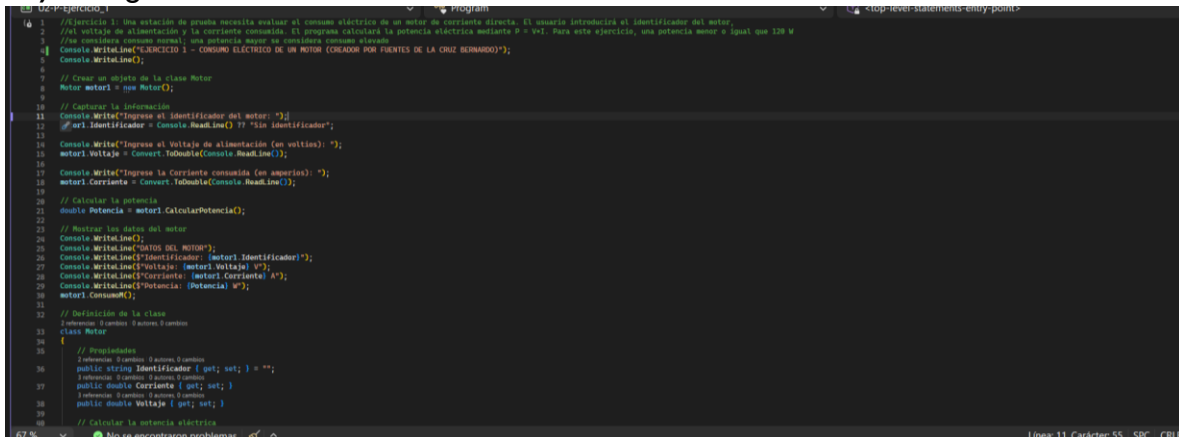


4) Tabla de POO.

Clases	motor
Objetos	motor 1
Propiedades	• Identificador (String/ Cadena) • Voltaje (Double/ decimal) • Corriente (Double/ decimal)
Métodos	• CalcularPotencia() • Consumo MC()

Parte 2: Desarrollo en Visual Studio:

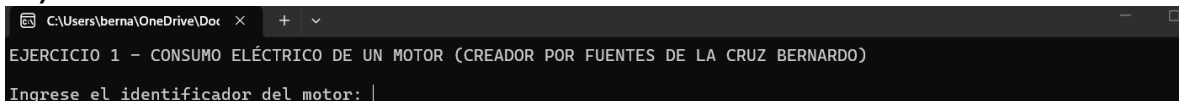
1) Código en C#.



```
1 //Ejercicio 1: Una estación de prueba necesita evaluar el consumo eléctrico de un motor de corriente directa. El usuario introducirá el identificador del motor;
2 //el voltaje de alimentación y la corriente consumida. El programa calculará la potencia eléctrica mediante  $P = V \cdot I$ . Para este ejercicio, una potencia menor o igual que 120 W
3 //se considera consumo normal; una potencia mayor se considera consumo elevado.
4 Console.WriteLine("EJERCICIO 1 - CONSUMO ELÉCTRICO DE UN MOTOR (CREADOR POR FUENTES DE LA CRUZ BERNARDO)");
5 Console.WriteLine();
6
7 // Crear un objeto de la clase Motor
8 Motor motor1 = new Motor();
9
10 // Capturar la información
11 Console.WriteLine("Ingrese el identificador del motor: ");
12 string identificador = Console.ReadLine();
13
14 Console.WriteLine("Ingrese el Voltaje de alimentación (en voltios): ");
15 motor1.Voltaje = Convert.ToDouble(Console.ReadLine());
16
17 Console.WriteLine("Ingrese la Corriente consumida (en amperios): ");
18 motor1.Corriente = Convert.ToDouble(Console.ReadLine());
19
20 // Calcular la potencia
21 double Potencia = motor1.CalcularPotencia();
22
23 // Mostrar los datos del motor
24 Console.WriteLine();
25 Console.WriteLine("DATOS DEL MOTOR");
26 Console.WriteLine($"Identificador: {motor1.Identificador}");
27 Console.WriteLine($"Voltaje: {motor1.Voltaje} V");
28 Console.WriteLine($"Corriente: {motor1.Corriente} A");
29 Console.WriteLine($"Potencia: {Potencia} W");
30 motor1.Consumo();
31
32 // Definición de la clase
33 namespace FuentesDeLaCruzBernardo
34 {
35     class Motor
36     {
37         // Propiedades
38         {
39             {
40                 {
41                     {
42                         {
43                             {
44                                 {
45                                     {
46                                     }
47                                 }
48                             }
49                         }
50                     }
51                 }
52             }
53         }
54     }
55 }
```

Fig. 1: Captura del Código del ejercicio 1.

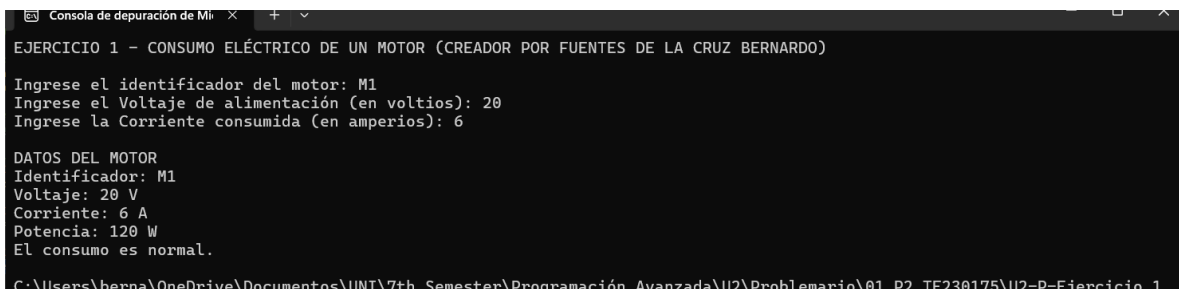
2) El encabezado en la consola.



```
C:\Users\berna\OneDrive\Doc x + v
EJERCICIO 1 - CONSUMO ELÉCTRICO DE UN MOTOR (CREADOR POR FUENTES DE LA CRUZ BERNARDO)
Ingrese el identificador del motor: |
```

Fig. 2: Captura de la consola mostrando el inicio del ejercicio 1.

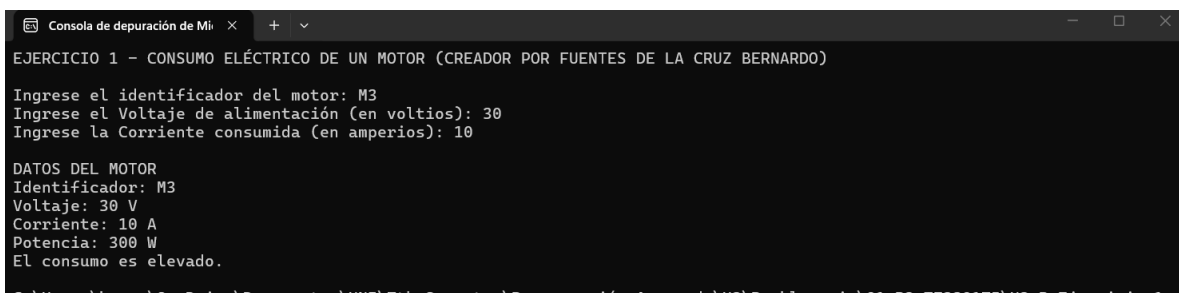
3) Evidencia de ejecución con dos Evidencias.



```
Consola de depuración de Mi x + v
EJERCICIO 1 - CONSUMO ELÉCTRICO DE UN MOTOR (CREADOR POR FUENTES DE LA CRUZ BERNARDO)
Ingrese el identificador del motor: M1
Ingrese el Voltaje de alimentación (en voltios): 20
Ingrese la Corriente consumida (en amperios): 6

DATOS DEL MOTOR
Identificador: M1
Voltaje: 20 V
Corriente: 6 A
Potencia: 120 W
El consumo es normal.
C:\Users\berna\OneDrive\Documents\UNTA\7th_Semester\Programación Avanzada\U2\Problemas\01_P2_TF230175\U2-P-Ejercicio_1
```

Fig. 3: Ejecución del código 1 (prueba 1).



```
Consola de depuración de Mi x + v
EJERCICIO 1 - CONSUMO ELÉCTRICO DE UN MOTOR (CREADOR POR FUENTES DE LA CRUZ BERNARDO)
Ingrese el identificador del motor: M3
Ingrese el Voltaje de alimentación (en voltios): 30
Ingrese la Corriente consumida (en amperios): 10

DATOS DEL MOTOR
Identificador: M3
Voltaje: 30 V
Corriente: 10 A
Potencia: 300 W
El consumo es elevado.
C:\Users\berna\OneDrive\Documents\UNTA\7th_Semester\Programación Avanzada\U2\Problemas\01_P2_TF230175\U2-P-Ejercicio_1
```

Fig. 4: Ejecución del código 1 (prueba 2).

4) Conclusión.

Se implementó una clase con encapsulamiento básico para modelar un motor de CD, calculando la potencia eléctrica mediante la relación $P = V \cdot I$ y evaluando el nivel de consumo mediante una estructura condicional simple.

Ejercicio 2:

Realizado por: Fuentes de la Cruz Bernardo.

Un cilindro neumático produce una fuerza aproximada mediante $F = P \cdot A$. El usuario proporcionará la presión en kPa y el área efectiva del pistón en cm². El programa convertirá la presión a Pa y el área a metros cuadrados. Después calculará la fuerza. La aplicación deberá indicar si la fuerza cumple un requerimiento mínimo introducido por el usuario.

Parte 1: Desarrollo a mano:

Parte 1 - Ejercicio 2.

1) Entradas, Procesos y Salidas

- Entradas: Fuerza Min, Presión Pa y Área cm.
- Procesos:
 - Conversión $P_a = \text{Presión hPa} \times 100$
 - $2 \text{ Área cm} / 10000 = \text{m}^2$.
 - Cálculo: $F = P_a \times m^2$.
 - Comparación: Evaluar si $F \geq \text{Fuerza Min}$.
- Salidas: Presión (Pa), Área (m²), Fuerza calculada (N), Fuerza requerida y decisión ("Cumple" o "No cumple").

2) Pseudocódigo

Inicio

Crear Objeto de Cilindro
 Leer Fuerza Min, Presión hPa, Área cm
 $\text{Área M}^2 = \text{Cilindro.Cálculo M}^2()$
 $\text{Presión Pa} = \text{Cilindro.Cálculo Pa}()$
 $\text{Fuerza} = \text{Cilindro.Cálculo F}()$
 Comparar Presión Pa, Área M², Fuerza, cálculo Fuerza Min
 Llamar Cilindro.Fuerza requerida()

// Fin //

Objeto Cilindro

Propiedades: Fuerza Min, Presión hPa, Área cm
 Métodos:
 $\text{Área M}^2 = \text{Área cm} / 10000$

Cálculo Pa()

$\text{Presión Pa} = \text{Presión hPa} \times 100$

Cálculo F()

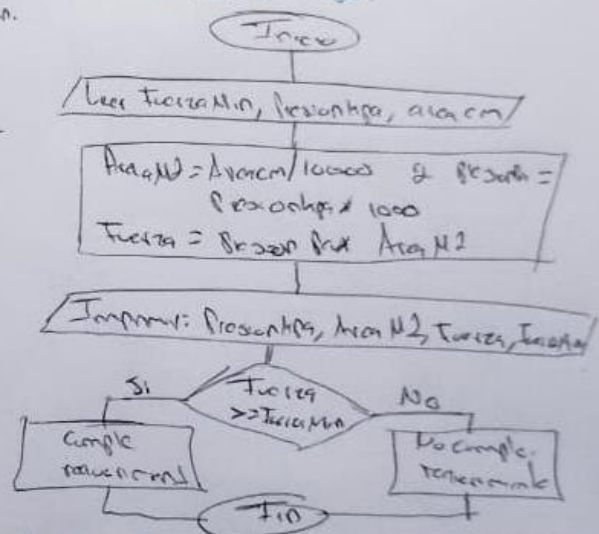
$\text{Fuerza} = \text{Cálculo Pa}() \times \text{Cálculo M}^2()$

Fuerza requerida()

Si $\text{cálculo F}() \geq \text{Fuerza Min}$ entonces
 Imprimir "Cumple con el requerimiento"
 Si no

Imprimir "No cumple con el requerimiento"

3) Diagrama de Flujo

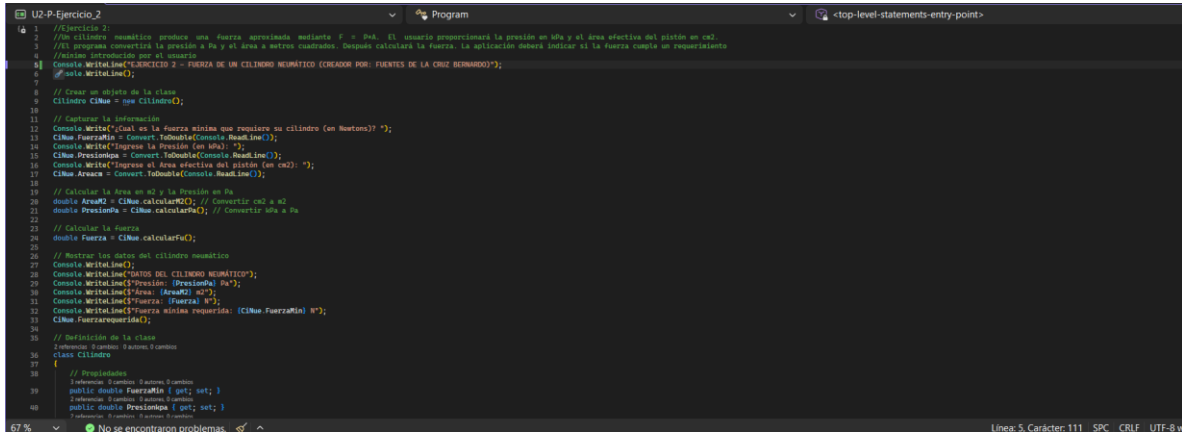


4) Tabla POO

Clases	Cilindro
Objetos	Cilindro
Propiedades	Fuerza Min (Real/double) Presión hPa (Real/double) Área cm (Real/double)
Métodos	Cálculo M²() Cálculo Pa() Cálculo F() Fuerza requerida()

Parte 2: Desarrollo en Visual Studio:

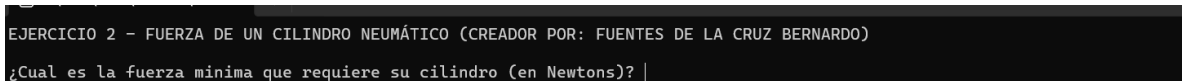
1) Código en C#.



```
1 // Ejercicio 2
2 // Un cilindro neumático produce una fuerza actuando mediante  $F = Pa$ . El usuario proporcionará la presión en kPa y el área efectiva del pistón en cm².
3 // El programa convertirá la presión a Pa y el área a metros cuadrados. Después calculará la fuerza. La aplicación deberá indicar si la fuerza cumple un requerimiento.
4 // Notas: Instrucción por el usuario
5 Console.WriteLine("EJERCICIO 2 - FUERZA DE UN CILINDRO NEUMÁTICO (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)");
6 Console.WriteLine();
7
8 // Crear un objeto de la clase
9 Cilindro Cilindro = new Cilindro();
10
11 // Capturar la información
12 Console.WriteLine("¿Cuál es la fuerza mínima que requiere su cilindro (en Newtons)? ");
13 Cilindro.FuerzaMin = Convert.ToInt32(Console.ReadLine());
14 Console.WriteLine("Ingrese la Presión (en kPa): ");
15 Cilindro.PresionKpa = Convert.ToDouble(Console.ReadLine());
16 Console.WriteLine("Ingrese el área efectiva del pistón (en cm²): ");
17 Cilindro.AreaCm2 = Convert.ToDouble(Console.ReadLine());
18
19 // Calcular la fuerza en N y la presión en Pa
20 double AreaM2 = Cilindro.calcularAreaM2(); // Convertir cm² a m²
21 double PresionPa = Cilindro.calcularPa(); // Convertir kPa a Pa
22
23 // Calcular la fuerza
24 double Fuerza = Cilindro.calcularFuerza();
25
26 // Mostrar los datos del cilindro neumático
27 Console.WriteLine();
28 Console.WriteLine("DATOS DEL CILINDRO NEUMÁTICO");
29 Console.WriteLine("Presión: " + PresionPa + " Pa");
30 Console.WriteLine("Área: " + AreaM2 + " m²");
31 Console.WriteLine("Fuerza: " + Fuerza + " N");
32 Console.WriteLine("Fuerza mínima requerida: " + Cilindro.FuerzaMin + " N");
33 Console.WriteLine();
34
35 // Definición de la clase
36 namespace FuentesBernardo
37 {
38     class Cilindro
39     {
40         // Propiedades
41         Informacion FuentesBernardo;
42         public double FuerzaMin { get; set; }
43         public double PresionKpa { get; set; }
44         public double AreaCm2 { get; set; }
45     }
46 }
```

Fig. 5: Captura del Código del ejercicio 2.

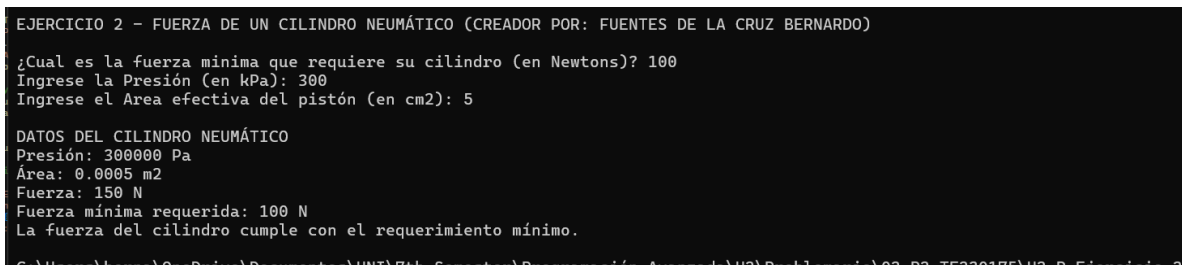
2) El encabezado en la consola.



```
EJERCICIO 2 - FUERZA DE UN CILINDRO NEUMÁTICO (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)
¿Cuál es la fuerza mínima que requiere su cilindro (en Newtons)? |
```

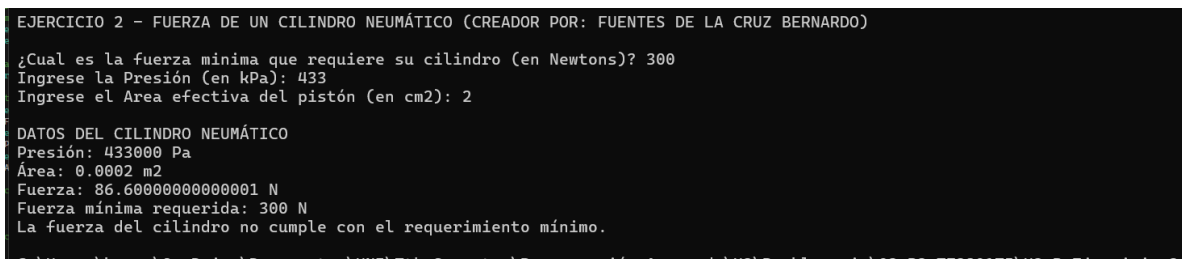
Fig. 6: Captura de la consola mostrando el inicio del ejercicio 2.

3) Evidencia de ejecución con dos Evidencias.



```
EJERCICIO 2 - FUERZA DE UN CILINDRO NEUMÁTICO (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)
¿Cuál es la fuerza mínima que requiere su cilindro (en Newtons)? 100
Ingrese la Presión (en kPa): 300
Ingrese el Área efectiva del pistón (en cm²): 5
DATOS DEL CILINDRO NEUMÁTICO
Presión: 300000 Pa
Área: 0.0005 m²
Fuerza: 150 N
Fuerza mínima requerida: 100 N
La fuerza del cilindro cumple con el requerimiento mínimo.
```

Fig. 7: Ejecución del código 2 (prueba 1).



```
EJERCICIO 2 - FUERZA DE UN CILINDRO NEUMÁTICO (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)
¿Cuál es la fuerza mínima que requiere su cilindro (en Newtons)? 300
Ingrese la Presión (en kPa): 433
Ingrese el Área efectiva del pistón (en cm²): 2
DATOS DEL CILINDRO NEUMÁTICO
Presión: 433000 Pa
Área: 0.0002 m²
Fuerza: 86.60000000000001 N
Fuerza mínima requerida: 300 N
La fuerza del cilindro no cumple con el requerimiento mínimo.
```

Fig. 8: Ejecución del código 2 (prueba 2).

4) Conclusión.

Se estructuraron métodos modulares para la conversión de unidades de presión y área, permitiendo calcular la fuerza neumática y determinar si el actuador satisface una restricción mínima de carga requerida por el usuario.

Ejercicio 3:

Realizado por: Fuentes de la Cruz Bernardo.

Durante una prueba se toman cinco mediciones de temperatura de un motor. El programa solicitará las cinco lecturas mediante un ciclo for y calculará el promedio. Si el promedio es menor o igual que 70 grados Celsius, el estado será NORMAL; si es mayor, mostrará ALERTA DE TEMPERATURA.

Parte 1: Desarrollo a mano:

Parte 1 - Ejercicio 3.

1) Entender, Procesar y Salidas:

- **Entradas:** Arreglo medicinas en 5 lecturas de temperatura en °C.
- **Procesar:**
 - Sumar los 5 valores acumulados mediante un ciclo: $suma = \sum medicinas[i]$.
 - Calcular el promedio: $Promedio = suma / 5$.
 - Evaluar si $Promedio \leq 70$.
- **Salidas:** Promedio en $\mathbb{Z}$ y estado de alerta ("Normal" o "Alerta de temperatura").

Si $calculoPromedio() \leq 70$ entonces

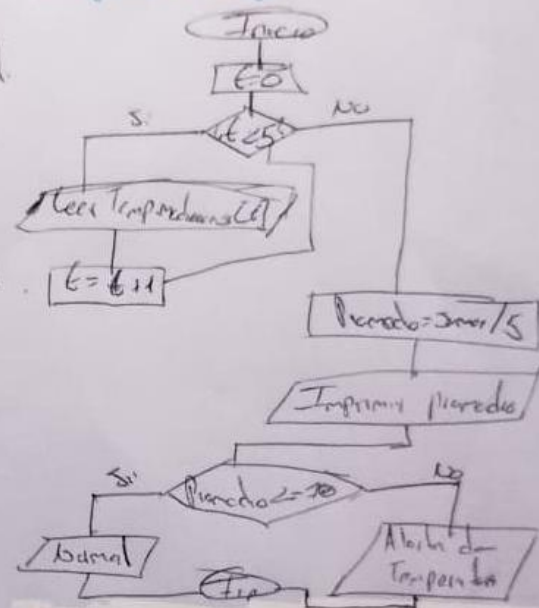
Imprimir: "Estado: Normal"

Sino

Imprimir: "Alerta de temperatura"

Fin

3) Diagrama de Flujo



2) Pseudocódigo:

Inicio

Crear Temp de Temperatura

Para $E=0$ hasta $E=5$; $E++$

Leer Temp. medicinas $[E]$

Fin para

Promedio = Temp. $calculoPromedio()$

Exibir "Promedio: "

Llamar Temp. $ObtenerTemperatura()$

//close//

close Temperatura

Requeridos: medicinas $[E]$

Calcular promedio

suma = 0

Para $E=0$ hasta $E=5$; $E++$

suma = suma + medicinas $[E]$

Fin para

promedio = suma / 5

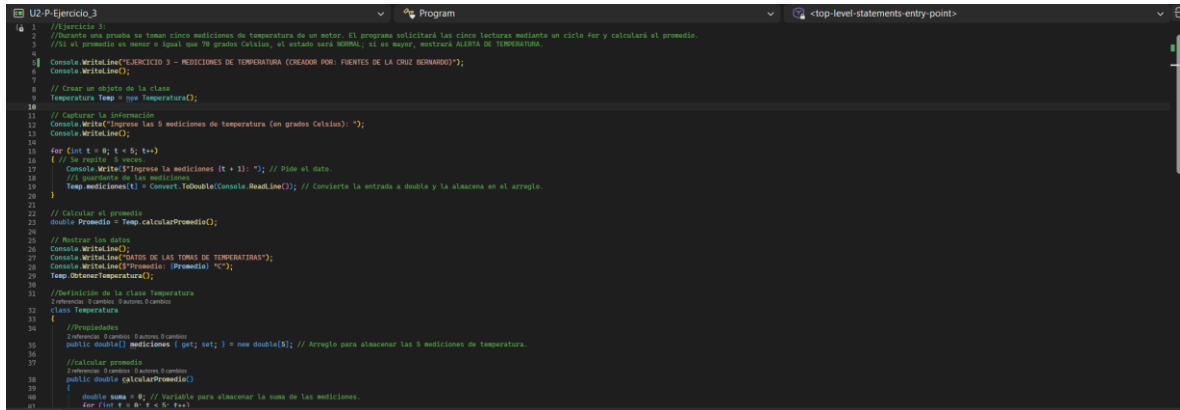
Obtener temperatura:

4) Tabla POO.

Clases	Temperatura
Objetos	Temp
Requeridos	medicinas (Arreglo int / double $[E]$)
Métodos	Calcular Promedio: Obtener Temperatura:

Parte 2: Desarrollo en Visual Studio:

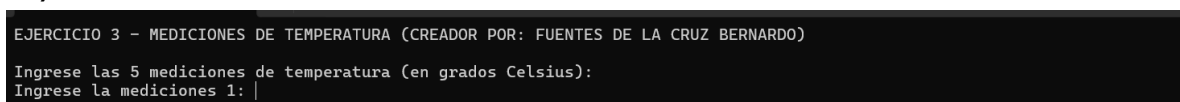
1) Código en C#.



```
1 //Alerta 1.  
2 //Después una prueba se toman cinco mediciones de temperatura de un motor. El programa solicitará las cinco lecturas mediante un ciclo for y calculará el promedio.  
3 //Si el promedio es menor a igual que 70 grados Celsius, el estado será NORMAL, si es mayor, mostrará ALERTA DE TEMPERATURA.  
4  
5 Console.WriteLine("EJERCICIO 3 - MEDICIONES DE TEMPERATURA (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)");  
6 Console.WriteLine();  
7  
8 // Crear un objeto de la clase  
9 Temperatura Temp = new Temperatura();  
10  
11 // Capturar la información  
12 Console.WriteLine("Ingrese las 5 mediciones de temperatura (en grados Celsius): ");  
13 Console.WriteLine();  
14  
15 for (int i = 0; i < 5; i++)  
16 {  
17     // Se pide el dato  
18     Console.WriteLine("Ingrese la mediciones (i + 1): "); // Pide el dato.  
19     // Leer el dato de las mediciones  
20     Temp.mediciones[i] = Convert.ToDouble(Console.ReadLine()); // Convierte la entrada a double y la almacena en el arreglo.  
21 }  
22  
23 // Calcular el promedio  
24 double Promedio = Temp.calcularPromedio();  
25  
26 // Mostrar los datos  
27 Console.WriteLine();  
28 Console.WriteLine("DATOS DE LAS TOMAS DE TEMPERATURAS");  
29 Console.WriteLine("Promedio: {Promedio} °C");  
30 Temp.ObtenerTemperatura();  
31  
32 //Definición de la clase Temperatura  
33 namespace Fuentes.Fuentes.Fuentes  
34 {  
35     //Propiedades  
36     double[] mediciones { get; set; } = new double[5]; // Arreglo para almacenar las 5 mediciones de temperatura.  
37  
38     //Calcular promedio  
39     double Promedio { get; set; }  
40     double suma = 0; // Variable para almacenar la suma de las mediciones.  
41     for (int i = 0; i < 5; i++)
```

Fig. 9: Captura del Código del ejercicio 3.

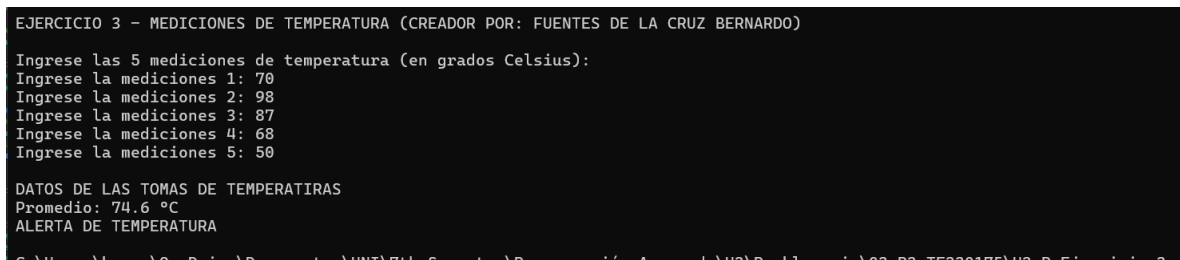
2) El encabezado en la consola.



```
EJERCICIO 3 - MEDICIONES DE TEMPERATURA (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)  
  
Ingrese las 5 mediciones de temperatura (en grados Celsius):  
Ingrese la mediciones 1: |
```

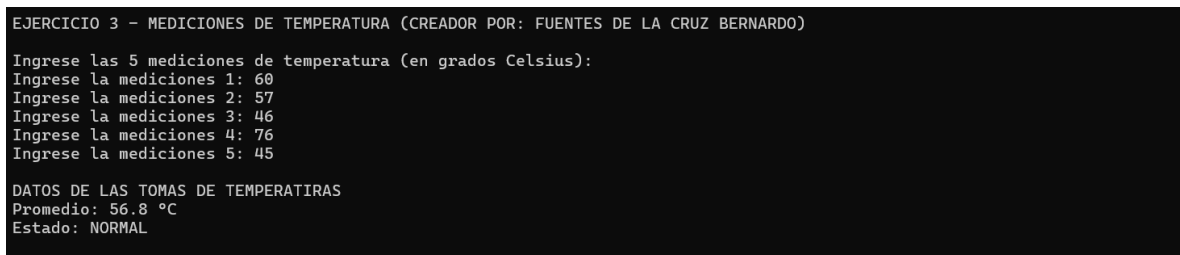
Fig. 10: Captura de la consola mostrando el inicio del ejercicio 3.

3) Evidencia de ejecución con dos Evidencias.



```
EJERCICIO 3 - MEDICIONES DE TEMPERATURA (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)  
  
Ingrese las 5 mediciones de temperatura (en grados Celsius):  
Ingrese la mediciones 1: 70  
Ingrese la mediciones 2: 98  
Ingrese la mediciones 3: 87  
Ingrese la mediciones 4: 68  
Ingrese la mediciones 5: 50  
  
DATOS DE LAS TOMAS DE TEMPERATURAS  
Promedio: 74.6 °C  
ALERTA DE TEMPERATURA
```

Fig. 11: Ejecución del código 3 (prueba 1).



```
EJERCICIO 3 - MEDICIONES DE TEMPERATURA (CREADOR POR: FUENTES DE LA CRUZ BERNARDO)  
  
Ingrese las 5 mediciones de temperatura (en grados Celsius):  
Ingrese la mediciones 1: 60  
Ingrese la mediciones 2: 57  
Ingrese la mediciones 3: 46  
Ingrese la mediciones 4: 76  
Ingrese la mediciones 5: 45  
  
DATOS DE LAS TOMAS DE TEMPERATURAS  
Promedio: 56.8 °C  
Estado: NORMAL
```

Fig. 12: Ejecución del código 3 (prueba 2).

4) Conclusión.

Se empleó un arreglo numérico y una estructura repetitiva for para gestionar múltiples lecturas de temperatura dentro de la clase, automatizando el cálculo del promedio y la activación de una alerta térmica.

Ejercicio 4:

Realizado por: Fuentes de la Cruz Bernardo.

Una batería debe cargarse hasta alcanzar 12.6 V. El usuario introducirá el voltaje inicial y el incremento de voltaje producido en cada ciclo. Mientras el voltaje sea menor que 12.6 V, el programa aumentará el valor, mostrará el número de ciclo y el voltaje actualizado. Si el último incremento supera el límite, el valor final deberá ajustarse a 12.6 V.

Parte 1: Desarrollo a mano:

EJERCICIO 4 - Parte 1

1) Identificar las entradas, el proceso y las salidas del sistema

Entradas: Voltaje Inicial (V_i), Incremento (r).

Proceso: Validar que Incremento > 0 .

Ciclo mientras Voltaje Inicial < 12.6 : incrementar Contador de ciclo ($ciclo + 1$), Sumar incremento ($Voltaje Inicial + Incremento$) y si supera 12.6 fijar Voltaje Inicial $= 12.6$.

Salidas: Mensaje de error (si incremento ≤ 0), reporte por ciclo (numero de ciclo y de voltaje actual) y aviso final de carga completa.

2) Pseudocódigo

Clase Bateria

Propiedades: Voltaje inicial (Real), Incremento (Real)

Método Cargar Bateria

Si Incremento ≤ 0 entonces

Escribir "Error: nuestro debe ser mayor que cero"

Sino

Ciclo $\leftarrow 0$

Mientras Voltaje Inicial ≤ 12.6 Hacer

Ciclo $\leftarrow$ ciclo + 1

Voltaje Inicial $\leftarrow$ Voltaje Inicial + Incremento

Si Voltaje Inicial > 12.6 entonces

Voltaje Inicial $\leftarrow 12.6$

Fin

Escribir "ciclo: ", ciclo, ", Voltaje actual: ", Voltaje Inicial

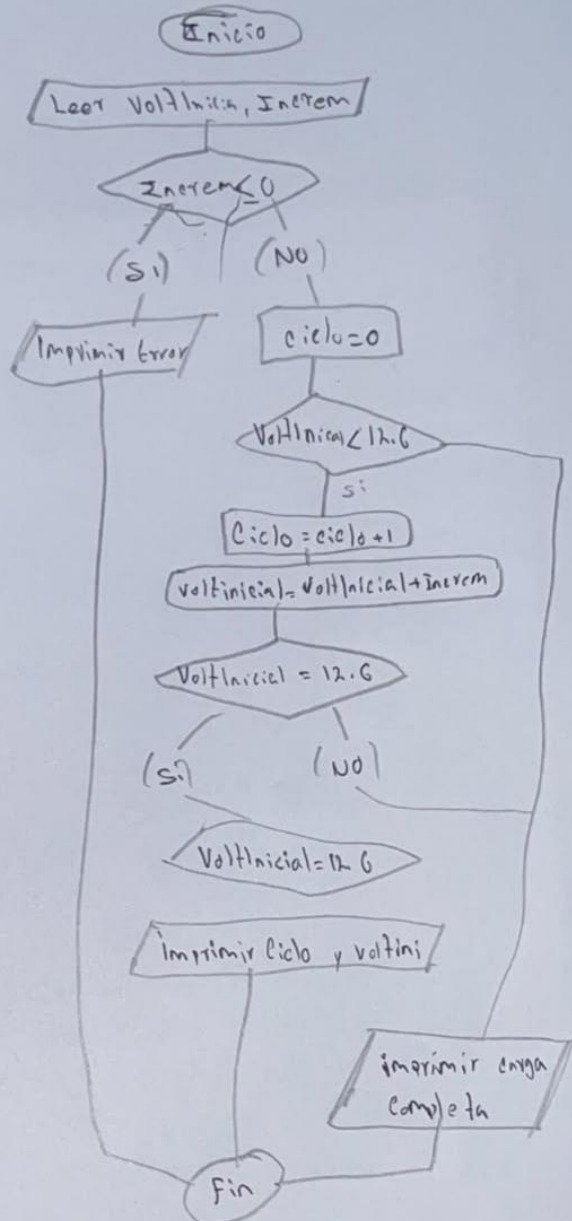
Fin mientras

Escribir: "La batería ha alcanzado su Voltaje máximo de 12.6V."

Fin si

Fin si.

3) Diagrama de Flujo

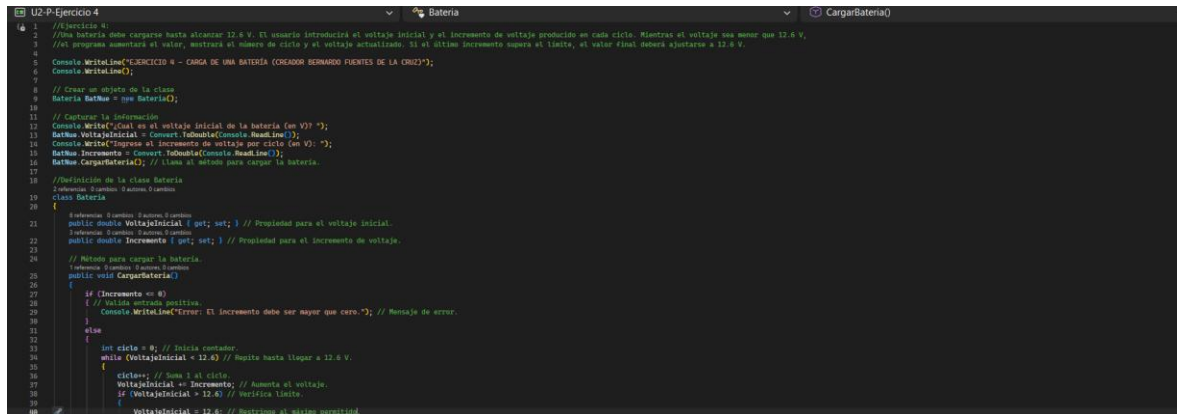


4) Tabla POO

clase	Bateria
objetos	BatNue
propiedades	Voltaje inicial Incremento
metodos	Cargar Bateria

Parte 2: Desarrollo en Visual Studio:

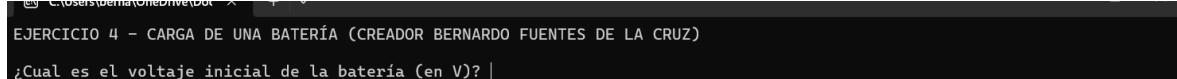
1) Código en C#.



```
1 //Ejercicio 4
2 //Una batería debe cargarse hasta alcanzar 12.6 V. El usuario introducirá el voltaje inicial y el incremento de voltaje producido en cada ciclo. Mientras el voltaje sea menor que 12.6 V,
3 //debe seguir aumentando el valor, mostrará el número de ciclos y el voltaje actualizado. Si el último incremento supera el límite, el valor final deberá ajustarse a 12.6 V.
4
5 Console.WriteLine("EJERCICIO 4 - CARGA DE UNA BATERÍA (CREADOR: BERNARDO FUENTES DE LA CRUZ)");
6 Console.WriteLine();
7
8 // Crear un objeto de la clase
9 Bateria batMan = new Bateria();
10
11 // Capturar la información
12 Console.WriteLine("¿Cual es el voltaje inicial de la batería (en V)? ");
13 batMan.VoltajeInicial = Convert.ToDouble(Console.ReadLine());
14 Console.WriteLine("Ingrese el incremento de voltaje por ciclo (en V): ");
15 batMan.Incremento = Convert.ToDouble(Console.ReadLine());
16 batMan.CargarBateria(); // Llama al método para cargar la batería.
17
18 //Definición de la clase Bateria
19 //Atributos: 0 voltaje, 0 ciclos, 0 incrementos
20 class Bateria
21 {
22     //Atributos: 0 voltaje, 0 ciclos, 0 incrementos
23     public double VoltajeInicial { get; set; } // Propiedad para el voltaje inicial.
24     //Atributos: 0 ciclos, 0 incrementos
25     public double Incremento { get; set; } // Propiedad para el incremento de voltaje.
26
27     // Método para cargar la batería.
28     //Atributos: 0 voltaje, 0 ciclos, 0 incrementos
29     public void CargarBateria()
30     {
31         if (Incremento == 0)
32         {
33             Console.WriteLine("Error: El incremento debe ser mayor que cero."); // Mensaje de error.
34         }
35         else
36         {
37             int ciclo = 0; // Inicio contador.
38             while (VoltajeInicial < 12.6) // Repite hasta llegar a 12.6 V.
39             {
40                 ciclo++; // Suma 1 al ciclo.
41                 VoltajeInicial += Incremento; // Aumenta el voltaje.
42                 Console.WriteLine("Ciclo {0}: Voltaje actual = {1:0.00} V", ciclo, VoltajeInicial);
43             }
44             VoltajeInicial = 12.6; // Redondeo al máximo permitido.
45         }
46     }
47 }
```

Fig. 13: Captura del Código del ejercicio 4.

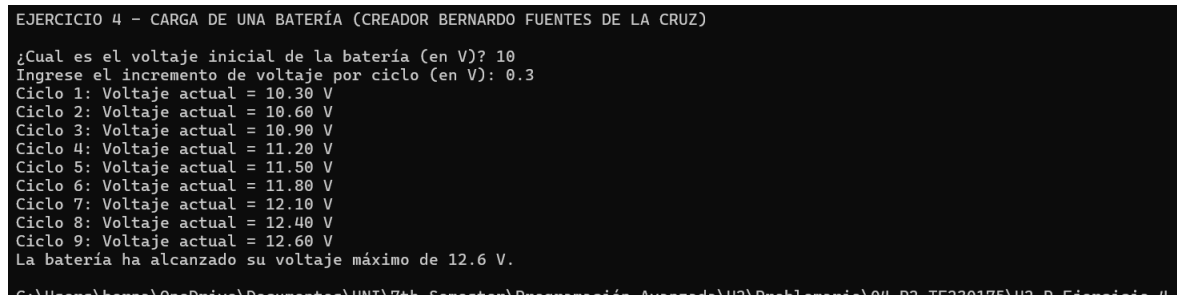
2) El encabezado en la consola.



```
EJERCICIO 4 - CARGA DE UNA BATERÍA (CREADOR BERNARDO FUENTES DE LA CRUZ)
¿Cual es el voltaje inicial de la batería (en V)? |
```

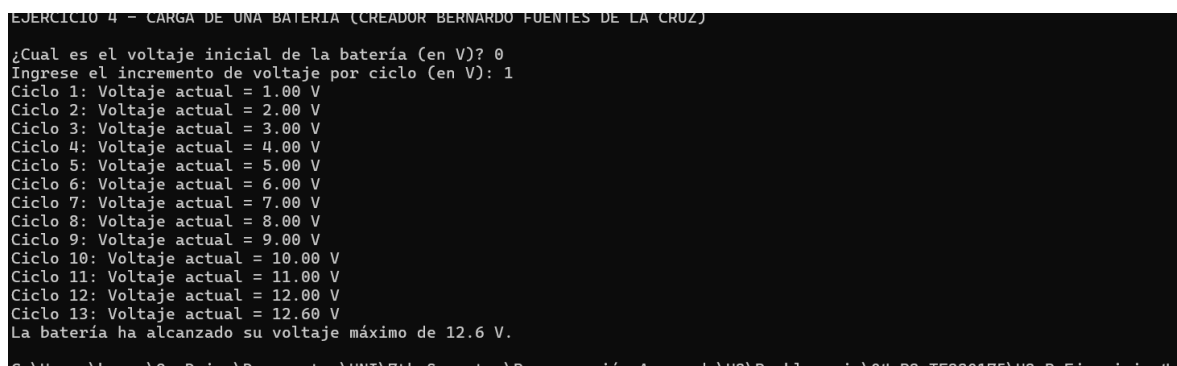
Fig. 14: Captura de la consola mostrando el inicio del ejercicio 4.

3) Evidencia de ejecución con dos Evidencias.



```
EJERCICIO 4 - CARGA DE UNA BATERÍA (CREADOR BERNARDO FUENTES DE LA CRUZ)
¿Cual es el voltaje inicial de la batería (en V)? 10
Ingrese el incremento de voltaje por ciclo (en V): 0.3
Ciclo 1: Voltaje actual = 10.30 V
Ciclo 2: Voltaje actual = 10.60 V
Ciclo 3: Voltaje actual = 10.90 V
Ciclo 4: Voltaje actual = 11.20 V
Ciclo 5: Voltaje actual = 11.50 V
Ciclo 6: Voltaje actual = 11.80 V
Ciclo 7: Voltaje actual = 12.10 V
Ciclo 8: Voltaje actual = 12.40 V
Ciclo 9: Voltaje actual = 12.60 V
La batería ha alcanzado su voltaje máximo de 12.6 V.
```

Fig. 15: Ejecución del código 4 (prueba 1).



```
EJERCICIO 4 - CARGA DE UNA BATERÍA (CREADOR BERNARDO FUENTES DE LA CRUZ)
¿Cual es el voltaje inicial de la batería (en V)? 0
Ingrese el incremento de voltaje por ciclo (en V): 1
Ciclo 1: Voltaje actual = 1.00 V
Ciclo 2: Voltaje actual = 2.00 V
Ciclo 3: Voltaje actual = 3.00 V
Ciclo 4: Voltaje actual = 4.00 V
Ciclo 5: Voltaje actual = 5.00 V
Ciclo 6: Voltaje actual = 6.00 V
Ciclo 7: Voltaje actual = 7.00 V
Ciclo 8: Voltaje actual = 8.00 V
Ciclo 9: Voltaje actual = 9.00 V
Ciclo 10: Voltaje actual = 10.00 V
Ciclo 11: Voltaje actual = 11.00 V
Ciclo 12: Voltaje actual = 12.00 V
Ciclo 13: Voltaje actual = 12.60 V
La batería ha alcanzado su voltaje máximo de 12.6 V.
```

Fig. 16: Ejecución del código 4 (prueba 2).

4) Conclusión.

Se encapsuló la lógica de control y validación dentro de un método de clase, utilizando un ciclo while para simular incrementos de voltaje paso a paso hasta alcanzar y limitar con precisión el umbral seguro de 12.6 V.

Ejercicio 5:

Realizado por: De La Fuente Chapuz Enrique.

Un servomotor puede moverse únicamente entre 0 y 180 grados. El usuario introducirá la posición actual y una posición objetivo. Mientras alguno de los valores esté fuera del intervalo permitido, deberá solicitarse nuevamente. Una vez que ambos valores sean válidos, el programa calculará el desplazamiento absoluto y determinará si el movimiento es hacia una posición mayor, hacia una posición menor o si no hay movimiento.

Parte 1: Desarrollo a mano:

EXERCICIO 5 - Parte 1

1) Identifica las entradas, el proceso y las salidas del programa.

Entradas: • Posición actual, posición objetivo, (ambos entre 0° y 180°)

Proceso: • Validar que $0 \leq \text{posición actual} \leq 180$, y $\text{posición objetivo} \leq 180$
 • Desplazamiento = [posición objetivo - posición actual]
 • Evaluar sentido: Si posición objetivo > posición actual ("Hacia posición mayor") si posición objetivo < posición actual ("Hacia posición menor"), si no ("sin movimiento")

2) Pseudocódigo.

Inicio

Crear el objeto servomotor

Repetir

Pedir y leer PosiciónActual

Hasta que $(\text{posición actual} \geq 0 \text{ y } \text{posición actual} \leq 180)$.

Repetir

Pedir y leer PosiciónObjetivo

Hasta que $\text{posición objetivo} \geq 0 \text{ y } \text{posición objetivo} \leq 180$

llamar al objeto servomotor

// Definir clase //

Propiedades: PosiciónActual, PosiciónObjetivo,

método: Calcular desplazamiento

Retornar $(\text{posición objetivo} - \text{posición actual})$

método determinar sentido

- Si $\text{posición objetivo} > \text{posición actual}$:

retornar "Hacia mayor posición"

- si no, si $\text{posición objetivo} < \text{posición actual}$:

retornar "Hacia posición menor"

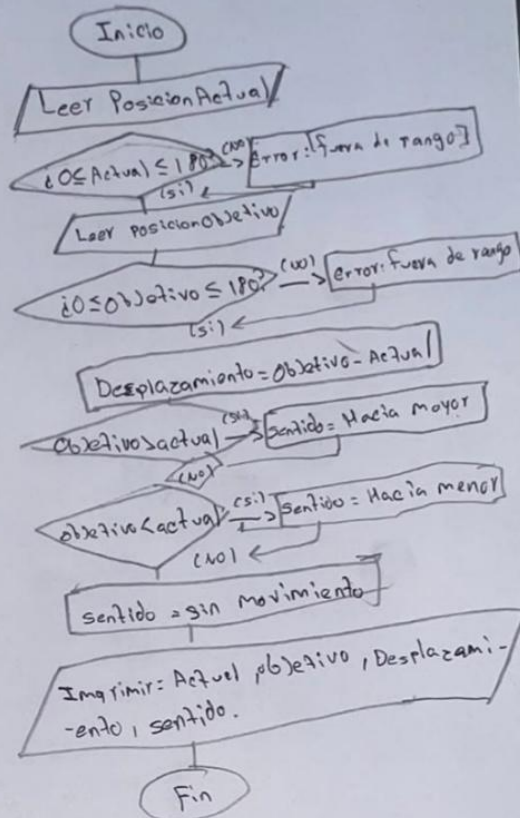
- Si no,

retornar "sin movimiento"

Imprimir posición actual, posición objetivo, desplazamiento (°), sentido.

Fin.

3) Diagrama de Flujo

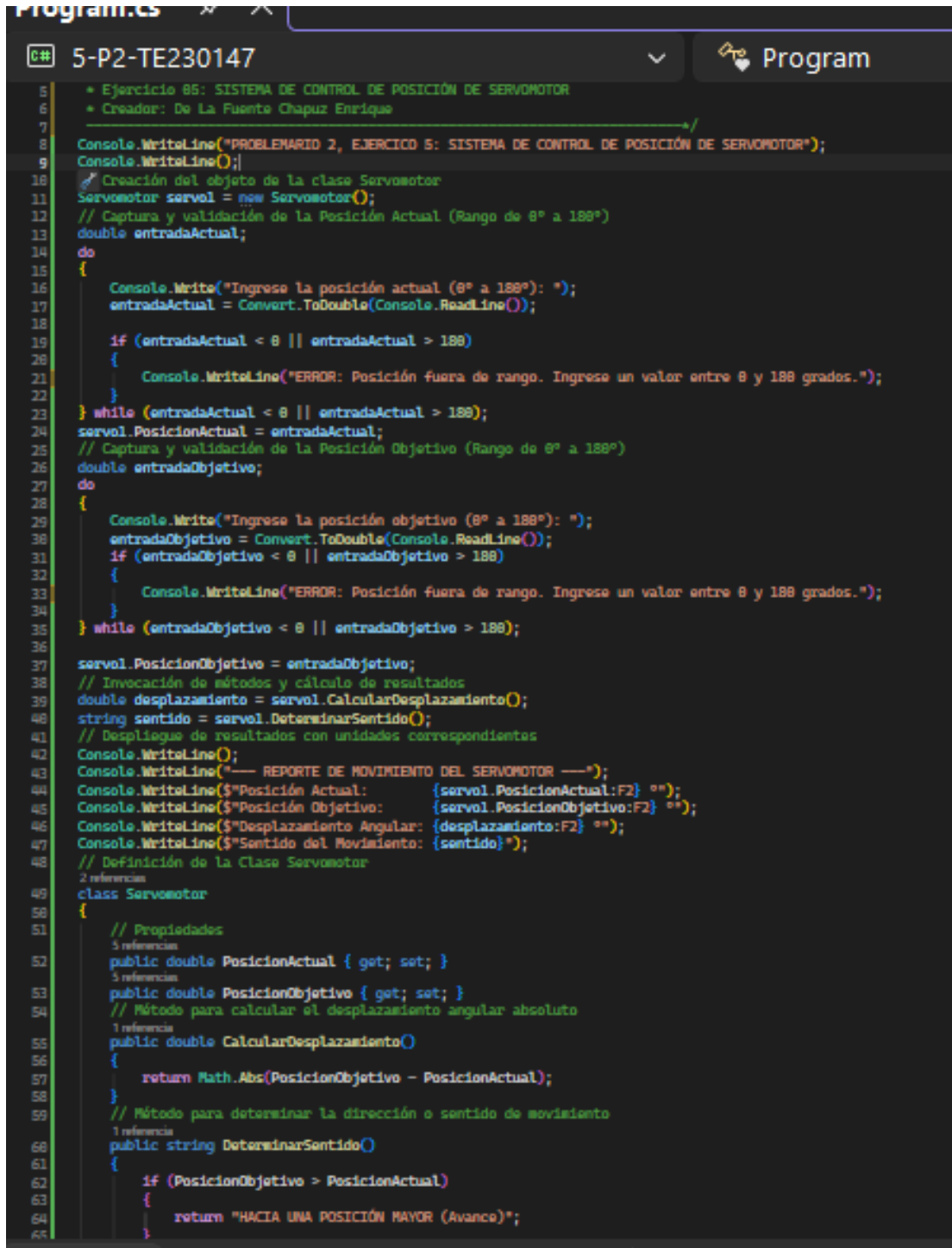


4) Tabla POO

Clases	servomotor
objetos	servo1
propiedades	posición actual posición objetivo
métodos	calcular desplazamiento determinar sentido.

Parte 2: Desarrollo en Visual Studio:

1) Código en C#.



```
Program.cs
C# 5-P2-TE230147 Program
5  * Ejercicio 05: SISTEMA DE CONTROL DE POSICIÓN DE SERVOMOTOR
6  * Creador: De La Fuente Chapuz Enrique
7
8  Console.WriteLine("PROBLEMATARIO 2, EJERCICIO 5: SISTEMA DE CONTROL DE POSICIÓN DE SERVOMOTOR");
9  Console.WriteLine();
10
11  // Creación del objeto de la clase Servomotor
12  Servomotor servol = new Servomotor();
13  // Captura y validación de la Posición Actual (Rango de 0° a 180°)
14  double entradaActual;
15  do
16  {
17      Console.Write("Ingrese la posición actual (0° a 180°): ");
18      entradaActual = Convert.ToDouble(Console.ReadLine());
19
20      if (entradaActual < 0 || entradaActual > 180)
21      {
22          Console.WriteLine("ERROR: Posición fuera de rango. Ingrese un valor entre 0 y 180 grados.");
23      }
24  } while (entradaActual < 0 || entradaActual > 180);
25  servol.PosicionActual = entradaActual;
26  // Captura y validación de la Posición Objetivo (Rango de 0° a 180°)
27  double entradaObjetivo;
28  do
29  {
30      Console.Write("Ingrese la posición objetivo (0° a 180°): ");
31      entradaObjetivo = Convert.ToDouble(Console.ReadLine());
32      if (entradaObjetivo < 0 || entradaObjetivo > 180)
33      {
34          Console.WriteLine("ERROR: Posición fuera de rango. Ingrese un valor entre 0 y 180 grados.");
35      }
36  } while (entradaObjetivo < 0 || entradaObjetivo > 180);
37
38  servol.PosicionObjetivo = entradaObjetivo;
39  // Invocación de métodos y cálculo de resultados
40  double desplazamiento = servol.CalcularDesplazamiento();
41  string sentido = servol.DeterminarSentido();
42  // Despliegue de resultados con unidades correspondientes
43  Console.WriteLine();
44  Console.WriteLine("— REPORTE DE MOVIMIENTO DEL SERVOMOTOR —");
45  Console.WriteLine($"Posición Actual: {servol.PosicionActual:F2} °");
46  Console.WriteLine($"Posición Objetivo: {servol.PosicionObjetivo:F2} °");
47  Console.WriteLine($"Desplazamiento Angular: {desplazamiento:F2} °");
48  Console.WriteLine($"Sentido del Movimiento: {sentido}");
49
50  // Definición de la Clase Servomotor
51  2 referencias
52  class Servomotor
53  {
54      // Propiedades
55      5 referencias
56      public double PosicionActual { get; set; }
57      5 referencias
58      public double PosicionObjetivo { get; set; }
59      // Método para calcular el desplazamiento angular absoluto
60      1 referencia
61      public double CalcularDesplazamiento()
62      {
63          return Math.Abs(PosicionObjetivo - PosicionActual);
64      }
65      // Método para determinar la dirección o sentido de movimiento
66      1 referencia
67      public string DeterminarSentido()
68      {
69          if (PosicionObjetivo > PosicionActual)
70          {
71              return "HACIA UNA POSICIÓN MAYOR (Avance)";
72          }
73      }
74  }
```

Fig. 17: Captura del Código del ejercicio 5.

2) El encabezado en la consola.

```
PROBLEMARIO 2, EJERCICO 5: SISTEMA DE CONTROL DE POSICIÓN DE SERVOMOTOR  
Ingrese la posición actual (0° a 180°): 0
```

Fig. 18: Captura de la consola mostrando el inicio del ejercicio 5.

3) Evidencia de ejecución con dos Evidencias.

```
PROBLEMARIO 2, EJERCICO 5: SISTEMA DE CONTROL DE POSICIÓN DE SERVOMOTOR  
  
Ingrese la posición actual (0° a 180°): 0  
Ingrese la posición objetivo (0° a 180°): 90  
  
--- REPORTE DE MOVIMIENTO DEL SERVOMOTOR ---  
Posición Actual:      0.00 °  
Posición Objetivo:    90.00 °  
Desplazamiento Angular: 90.00 °  
Sentido del Movimiento: HACIA UNA POSICIÓN MAYOR (Avance)  
  
C:\Users\enriq\OneDrive\Escritorio\PROGRAM AVANZ 7C\5-P2-TE230147\bin\Debug\net10.0\5-P2-TE230147.exe (proceso 25208) se  
cerró con el código 0 (0x0).  
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->  
Cerrar la consola automáticamente al detenerse la depuración.  
Presione cualquier tecla para cerrar esta ventana. . .|
```

Fig. 19: Ejecución del código 5 (prueba 1).

```
PROBLEMARIO 2, EJERCICO 5: SISTEMA DE CONTROL DE POSICIÓN DE SERVOMOTOR  
  
Ingrese la posición actual (0° a 180°): 180  
Ingrese la posición objetivo (0° a 180°): 180  
  
--- REPORTE DE MOVIMIENTO DEL SERVOMOTOR ---  
Posición Actual:      180.00 °  
Posición Objetivo:    180.00 °  
Desplazamiento Angular: 0.00 °  
Sentido del Movimiento: SIN MOVIMIENTO (Posición alcanzada)  
  
C:\Users\enriq\OneDrive\Escritorio\PROGRAM AVANZ 7C\5-P2-TE230147\bin\Debug\net10.0\5-P2-TE230147.exe (proceso 32348) se  
cerró con el código 0 (0x0).  
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->  
Cerrar la consola automáticamente al detenerse la depuración.  
Presione cualquier tecla para cerrar esta ventana. . .|
```

Fig. 20: Ejecución del código 5 (prueba 2).

4) Conclusión.

En esta práctica se modeló el comportamiento cinemático angular de un servomotor bajo el paradigma de Programación Orientada a Objetos. Mediante el encapsulamiento de las posiciones actual y objetivo dentro de la clase Servomotor, se garantizó que los cálculos de desplazamiento absoluto y la determinación del sentido de giro se resolvieran de forma modular a través de métodos especializados. Asimismo, la integración de ciclos de validación antes del procesamiento aseguró el respeto a los límites físicos y de seguridad del actuador (0° a 180°), evitando condiciones fuera de rango o errores operacionales en el sistema mecatrónico.

Ejercicio 6:

Realizado por: De La Fuente Chapuz Enrique.

Una celda automatizada inspecciona un lote de diez piezas. Para cada pieza, el usuario introducirá 1 si la pieza es correcta o 0 si es defectuosa. Cualquier otro valor deberá rechazarse y solicitarse nuevamente. Al terminar, el programa mostrará las cantidades, el porcentaje de piezas correctas y el estado del lote. Un lote con al menos 90 por ciento de piezas correctas será ACEPTADO; de lo contrario será RECHAZADO.

Parte 1: Desarrollo a mano:

EJERCICIO 6-Parte 1

1) Identificar las entradas, el proceso y las salidas del problema

Entradas: Estado de 10 piezas (1 = Correcta, 0 = Defectuosa).

Proceso: Ciclo de 1 a 10 para capturar cada pieza

Validar que la entrada sea 1, 0.

Contar piezas correctas y defectuosas

Porcentaje = $(\text{Correctas} / 10) \times 100$,

Evaluar: si porcentaje $\geq 90\%$ (aceptado) o no (rechazado).

Salidas: Cantidad de correctas, defectuosas
Porcentaje de calidad, estado del lote

2) Pseudo Código

Inicio

Crear el objeto local

Para $i = 1$ hasta 10 con paso 1:

Repetir: leer valor (1 o 0)

Hasta que ($\text{valor} = 1$ o $\text{valor} = 0$)

lote1.RegistrarPieza(valor).

// Definir clase Lote piezas //

Propiedades: Total piezas (10), piezas correctas, piezas defectuosas.

Método registrar pieza (v)

si $v = 1$: correcta, si $v = 0$: defectuosa

Método calcular promedio

Retornar $(\text{Correctas} / 10) \times 100$

Método determinar estado

Retornar (calcular porcentaje ≥ 90 = "aceptado")

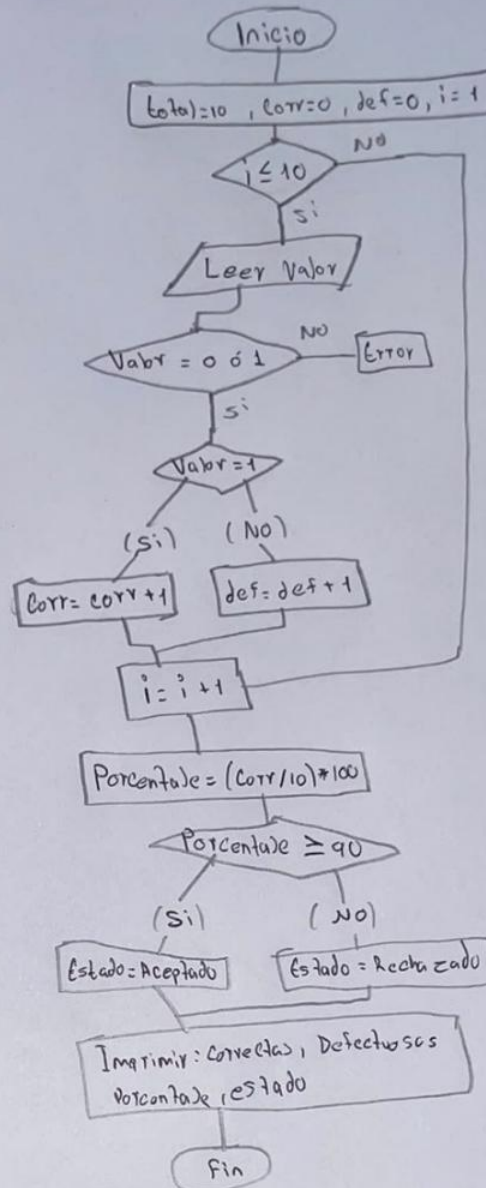
si no: "rechazado"

Imprimir:

correctas, defectuosas, calcular porcentaje, determinar estado.

Fin

3) Diagrama de Flujo



4) Tabla POO

Clases	Lote Piezas
objetos	Lote1
Propiedades	Total piezas, Piezas correctas y piezas defectuosas
métodos	Registrar pieza (estado) Calcular porcentaje Determinar estado

Parte 2: Desarrollo en Visual Studio:

1) Código en C#.

```
C# 6-P2-TE230147 Program
4  * Ejercicio 06: CELDA AUTOMATIZADA - INSPECCIÓN DE CALIDAD DE LOTE
5  * Creador: De La Fuente Chapuz Enrique
6
7  using System;
8
9  Console.WriteLine("EJERCICIO 06: CELDA AUTOMATIZADA - INSPECCIÓN DE CALIDAD DE LOTE");
10 Console.WriteLine("=====");
11 Console.WriteLine();
12
13 // 1. Instanciación del objeto LotePiezas
14 LotePiezas lote1 = new LotePiezas();
15
16 // 2. Registro iterativo de las 18 piezas con ciclo for
17 for (int i = 1; i <= lote1.TotalPiezas; i++)
18 {
19     int lectura;
20     do
21     {
22         Console.WriteLine($"Pieza [{i}/{lote1.TotalPiezas}] - Ingrese estado (1 = Correcta, 0 = Defectuosa): ");
23         lectura = Convert.ToInt32(Console.ReadLine());
24
25         if (lectura != 0 && lectura != 1)
26         {
27             Console.WriteLine("[ERROR] Valor inválido. Únicamente se permite ingresar 1 (Correcta) o 0 (Defectuosa).");
28         }
29     } while (lectura != 0 && lectura != 1);
30
31     // Registro dentro del objeto
32     lote1.RegistrarPieza(lectura);
33 }
34
35 // 3. Obtención de métricas mediante los métodos del objeto
36 double porcentajeEfectividad = lote1.CalcularPorcentajeCorrectas();
37 string dictamen = lote1.DeterminarEstadoLote();
38
39 // 4. Salida de resultados con unidades correspondientes
40 Console.WriteLine();
41 Console.WriteLine("— REPORTE FINAL DE INSPECCIÓN —");
42 Console.WriteLine($"Total inspeccionado: {lote1.TotalPiezas} piezas");
43 Console.WriteLine($"Piezas Correctas: {lote1.PiezasCorrectas} piezas");
44 Console.WriteLine($"Piezas Defectuosas: {lote1.PiezasDefectuosas} piezas");
45 Console.WriteLine($"Porcentaje Conforme: {porcentajeEfectividad:F2} %");
46 Console.WriteLine($"Dictamen del Lote: {dictamen}");
47
48
49 // Definición de la Clase LotePiezas
50 2 referencias
51 class LotePiezas
52 {
53     // Propiedades
54     3 referencias
55     public int TotalPiezas { get; } = 18;
56     3 referencias
57     public int PiezasCorrectas { get; private set; } = 0;
58     2 referencias
59     public int PiezasDefectuosas { get; private set; } = 0;
60
61     // Método para registrar la inspección de cada pieza
62     1 referencia
63     public void RegistrarPieza(int estado)
64     {
65         if (estado == 1)
66         {
67             PiezasCorrectas++;
68         }
69         else if (estado == 0)
70         {
71             PiezasDefectuosas++;
72         }
73     }
74 }
```

Fig. 21: Captura del Código del ejercicio 6.

2) El encabezado en la consola.

```
EJERCICIO 06: CELDA AUTOMATIZADA - INSPECCIÓN DE CALIDAD DE LOTE
=====
Pieza [1/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
```

Fig. 22: Captura de la consola mostrando el inicio del ejercicio 6.

3) Evidencia de ejecución con dos Evidencias.

```
EJERCICIO 06: CELDA AUTOMATIZADA - INSPECCIÓN DE CALIDAD DE LOTE
=====
Pieza [1/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [2/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [3/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [4/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [5/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [6/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [7/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [8/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [9/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [10/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1

--- REPORTE FINAL DE INSPECCIÓN ---
Total inspeccionado: 10 piezas
Piezas Correctas: 10 piezas
Piezas Defectuosas: 0 piezas
Porcentaje Conforme: 100.00 %
Dictamen del Lote: LOTE ACEPTADO
```

Fig. 23: Ejecución del código 6 (prueba 1).

```
Consola de depuración de I X + v
EJERCICIO 06: CELDA AUTOMATIZADA - INSPECCIÓN DE CALIDAD DE LOTE
=====
Pieza [1/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [2/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [3/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [4/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 0
Pieza [5/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 0
Pieza [6/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 0
Pieza [7/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 0
Pieza [8/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [9/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1
Pieza [10/10] - Ingrese estado (1 = Correcta, 0 = Defectuosa): 1

--- REPORTE FINAL DE INSPECCIÓN ---
Total inspeccionado: 10 piezas
Piezas Correctas: 6 piezas
Piezas Defectuosas: 4 piezas
Porcentaje Conforme: 60.00 %
Dictamen del Lote: LOTE RECHAZADO
```

Fig. 24: Ejecución del código 6 (prueba 2).

4) Conclusión.

La implementación de la clase LotePiezas permitió simular una estación automatizada de control de calidad, centralizando la lógica del conteo y la evaluación de piezas conformes y defectuosas. El uso de un ciclo for combinado con la validación de entradas binarias garantizó un registro determinista de las diez muestras. Por otro lado, delegar el cálculo del porcentaje de efectividad y el dictamen final ("ACEPTADO" o "RECHAZADO") a métodos internos del objeto reforzó la cohesión del código, previniendo posibles errores de división entre cero

y proporcionando un diagnóstico confiable para el control del proceso de manufactura.

Ejercicio 7:

Realizado por: De La Fuente Chapuz Enrique.

Un brazo robótico realiza un ciclo de tomar, trasladar y soltar una pieza. El usuario introducirá el tiempo de cada etapa en segundos y el número de ciclos que desea simular. El programa calculará el tiempo de un ciclo y utilizará un ciclo for para mostrar el tiempo acumulado después de cada repetición. Finalmente mostrará el tiempo total de producción.

Parte 1: Desarrollo a mano:

EJERCICIO 7 - Parte 1

1) Identificar las entradas, el proceso y las salidas del problema

Entradas: Tiempo tomar (s), tiempo trasladar (s), tiempo soltar (s), Numero de ciclos

Proceso: Tiempo por ciclo = tomar + trasladar + soltar
Ciclo de 1 a numero de ciclos para calcular el tiempo acumulado de cada repetición
Acumulado = ix tiempo de ciclo
Tiempo total = No. de ciclos por tiempo

Salidas: tiempo de un ciclo (s), tiempo acumulado por ciclo (s), tiempo de reproducción (s).

2) Pseudocódigo:

Inicio

Crear el objeto robot1

Pedir y leer: tiempo tomar, tiempo trasladar, tiempo soltar, numero ciclos.

llamar al objeto robot1

//Definición de clase BrazoRobótico//

Propiedades: tiempo tomar, tiempo trasladar, tiempo soltar, numero ciclos

Método: Calcular tiempo ciclo:

Retornar: tiempo tomar + tiempo trasladar + tiempo soltar

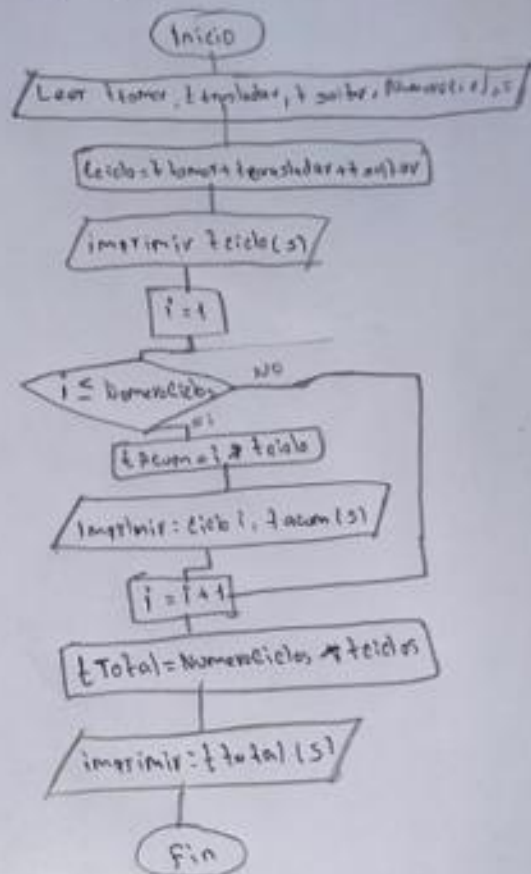
Método: Calcular tiempo total:

retornar: numero ciclos * calcular tiempo ciclo

Lecho = robot1. calcular tiempo ciclo
imprimir: tiempo de un ciclo, y ciclo "s"

Fin

3) Diagrama de flujo

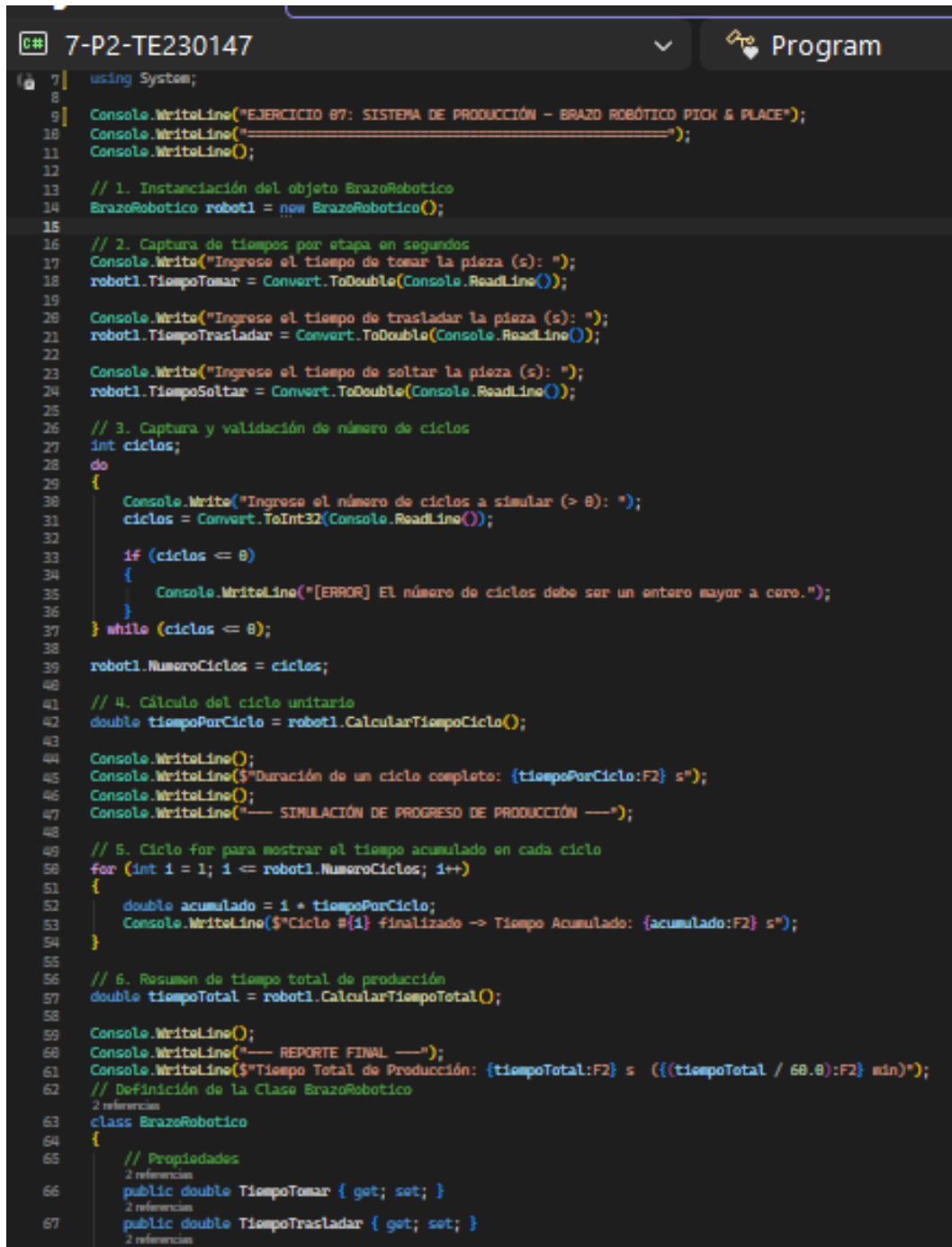


4) Tabla POO

Clases	BrazoRobótico
objetos	Robot1
Propiedades	tiempo tomar, tiempo trasladar, tiempo soltar, numero ciclos
Métodos	Calcular tiempo ciclo calcular tiempo total

Parte 2: Desarrollo en Visual Studio:

1) Código en C#.



```
7 | using System;
8 |
9 | Console.WriteLine("EJERCICIO 07: SISTEMA DE PRODUCCIÓN - BRAZO ROBÓTICO PICK & PLACE");
10 | Console.WriteLine("=====");
11 | Console.WriteLine();
12 |
13 | // 1. Instanciación del objeto BrazoRobotico
14 | BrazoRobotico robot1 = new BrazoRobotico();
15 |
16 | // 2. Captura de tiempos por etapa en segundos
17 | Console.Write("Ingrese el tiempo de tomar la pieza (s): ");
18 | robot1.TiempoTomar = Convert.ToDouble(Console.ReadLine());
19 |
20 | Console.Write("Ingrese el tiempo de trasladar la pieza (s): ");
21 | robot1.TiempoTrasladar = Convert.ToDouble(Console.ReadLine());
22 |
23 | Console.Write("Ingrese el tiempo de soltar la pieza (s): ");
24 | robot1.TiempoSoltar = Convert.ToDouble(Console.ReadLine());
25 |
26 | // 3. Captura y validación de número de ciclos
27 | int ciclos;
28 | do
29 | {
30 |     Console.Write("Ingrese el número de ciclos a simular (> 0): ");
31 |     ciclos = Convert.ToInt32(Console.ReadLine());
32 |
33 |     if (ciclos <= 0)
34 |     {
35 |         Console.WriteLine("[ERROR] El número de ciclos debe ser un entero mayor a cero.");
36 |     }
37 | } while (ciclos <= 0);
38 |
39 | robot1.NumeroCiclos = ciclos;
40 |
41 | // 4. Cálculo del ciclo unitario
42 | double tiempoPorCiclo = robot1.CalcularTiempoCiclo();
43 |
44 | Console.WriteLine();
45 | Console.WriteLine($"Duración de un ciclo completo: {tiempoPorCiclo:F2} s");
46 | Console.WriteLine();
47 | Console.WriteLine("— SIMULACIÓN DE PROGRESO DE PRODUCCIÓN —");
48 |
49 | // 5. Ciclo for para mostrar el tiempo acumulado en cada ciclo
50 | for (int i = 1; i <= robot1.NumeroCiclos; i++)
51 | {
52 |     double acumulado = i * tiempoPorCiclo;
53 |     Console.WriteLine($"Ciclo #{i} finalizado -> Tiempo Acumulado: {acumulado:F2} s");
54 | }
55 |
56 | // 6. Resumen de tiempo total de producción
57 | double tiempoTotal = robot1.CalcularTiempoTotal();
58 |
59 | Console.WriteLine();
60 | Console.WriteLine("— REPORTE FINAL —");
61 | Console.WriteLine($"Tiempo Total de Producción: {tiempoTotal:F2} s ({(tiempoTotal / 60.0):F2} min)");
62 | // Definición de la Clase BrazoRobotico
63 | 2 referencias
64 | class BrazoRobotico
65 | {
66 |     // Propiedades
67 |     2 referencias
68 |     public double TiempoTomar { get; set; }
69 |     2 referencias
70 |     public double TiempoTrasladar { get; set; }
71 |     2 referencias
```

Fig. 25: Captura del Código del ejercicio 7.

2) El encabezado en la consola.

```
EJERCICIO 07: SISTEMA DE PRODUCCIÓN - BRAZO ROBÓTICO PICK & PLACE
=====

Ingrese el tiempo de tomar la pieza (s): 10
```

Fig. 26: Captura de la consola mostrando el inicio del ejercicio 7.

3) Evidencia de ejecución con dos Evidencias.

```
EJERCICIO 07: SISTEMA DE PRODUCCIÓN - BRAZO ROBÓTICO PICK & PLACE
=====

Ingrese el tiempo de tomar la pieza (s): 10
Ingrese el tiempo de trasladar la pieza (s): 5
Ingrese el tiempo de soltar la pieza (s): 2
Ingrese el número de ciclos a simular (> 0): 3

Duración de un ciclo completo: 17.00 s

--- SIMULACIÓN DE PROGRESO DE PRODUCCIÓN ---
Ciclo #1 finalizado -> Tiempo Acumulado: 17.00 s
Ciclo #2 finalizado -> Tiempo Acumulado: 34.00 s
Ciclo #3 finalizado -> Tiempo Acumulado: 51.00 s

--- REPORTE FINAL ---
Tiempo Total de Producción: 51.00 s (0.85 min)
```

Fig. 27: Ejecución del código 7 (prueba 1).

```
EJERCICIO 07: SISTEMA DE PRODUCCIÓN - BRAZO ROBÓTICO PICK & PLACE
=====

Ingrese el tiempo de tomar la pieza (s): 20
Ingrese el tiempo de trasladar la pieza (s): 3
Ingrese el tiempo de soltar la pieza (s): 1
Ingrese el número de ciclos a simular (> 0): 5

Duración de un ciclo completo: 24.00 s

--- SIMULACIÓN DE PROGRESO DE PRODUCCIÓN ---
Ciclo #1 finalizado -> Tiempo Acumulado: 24.00 s
Ciclo #2 finalizado -> Tiempo Acumulado: 48.00 s
Ciclo #3 finalizado -> Tiempo Acumulado: 72.00 s
Ciclo #4 finalizado -> Tiempo Acumulado: 96.00 s
Ciclo #5 finalizado -> Tiempo Acumulado: 120.00 s

--- REPORTE FINAL ---
Tiempo Total de Producción: 120.00 s (2.00 min)
```

Fig. 28: Ejecución del código 7 (prueba 2).

4) Conclusión.

El desarrollo de la clase BrazoRobotico demostró la utilidad de la POO en la planificación y análisis temporal de sistemas industriales tipo Pick and Place. Al abstraer las tres fases operativas (tomar, trasladar y soltar) como propiedades del objeto, fue posible calcular el tiempo de ciclo unitario y proyectar con precisión el tiempo acumulado mediante un ciclo iterativo. Esto evidencia cómo la modularidad del software facilita el monitoreo secuencial de cada repetición y permite calcular el tiempo total de producción en unidades físicas normalizadas (s y min), aspectos esenciales para la optimización y sincronización de celdas robóticas.

Ejercicio 8:

Realizado por: Hernández Domínguez Bryan.

Un tanque recibe una cantidad constante de líquido en cada ciclo. El usuario introducirá la capacidad máxima, el volumen inicial y el volumen agregado por ciclo. Mientras el tanque no esté lleno, el programa actualizará el volumen, calculará el porcentaje de llenado y mostrará ambos datos. El volumen nunca deberá superar la capacidad.

Parte 1: Desarrollo a mano:

EJERCICIO 8

1. Identificar claramente los entrados, el proceso y las salidas del problema
- Entrados
 - Capacidad máxima del tanque
 - Volumen inicial
 - Volumen agregado por ciclos
- Proceso
 - Ingresar los valores de Capacidad máxima, Volumen actual y Volumen agregado
 - Validar que Volumen a agregar sea mayor que 0
 - Calcular el volumen y porcentaje de llenado inicial (Ciclo = 0)
 - Verificar Volumen Actual < Capacidad
 - Incrementar el ciclo (Ciclo = Ciclo + 1)
 - Sumar el volumen agregado (Volumen actual = Volumen actual + Volumen agregado)
 - Si Volumen actual > Capacidad, mostrar Volumen actual = Capacidad
 - Calcular porcentaje = $(\text{Volumen actual} / \text{Capacidad}) \times 100$
 - Imprimir ciclo, volumen actual (L) y porcentaje (P.)
 - Mostrar mensaje indicando que se alcanza capacidad máxima.
- Salidas
 - Mostrar el estado inicial con su volumen y porcentaje
 - Mostrar en cada iteración siguiente el número de ciclo, volumen actual y porcentaje
 - Mostrar mensaje de confirmación final

2) Pseudo código

```

Inicio
  Leer Capacidad
  Leer Volumen Actual
  Leer Volumen Agregar

Si Volumen Agregar <= 0 entonces
  Mostrar "Error: El volumen a agregar debe ser mayor a cero"
Si no
  Ciclo = 0
  Porcentaje = (Volumen Actual / Capacidad) * 100
  Mostrar "Ciclo " + Ciclo + ": Volumen actual = " + Volumen Actual + " L | Llenado = " + porcentaje + "% "

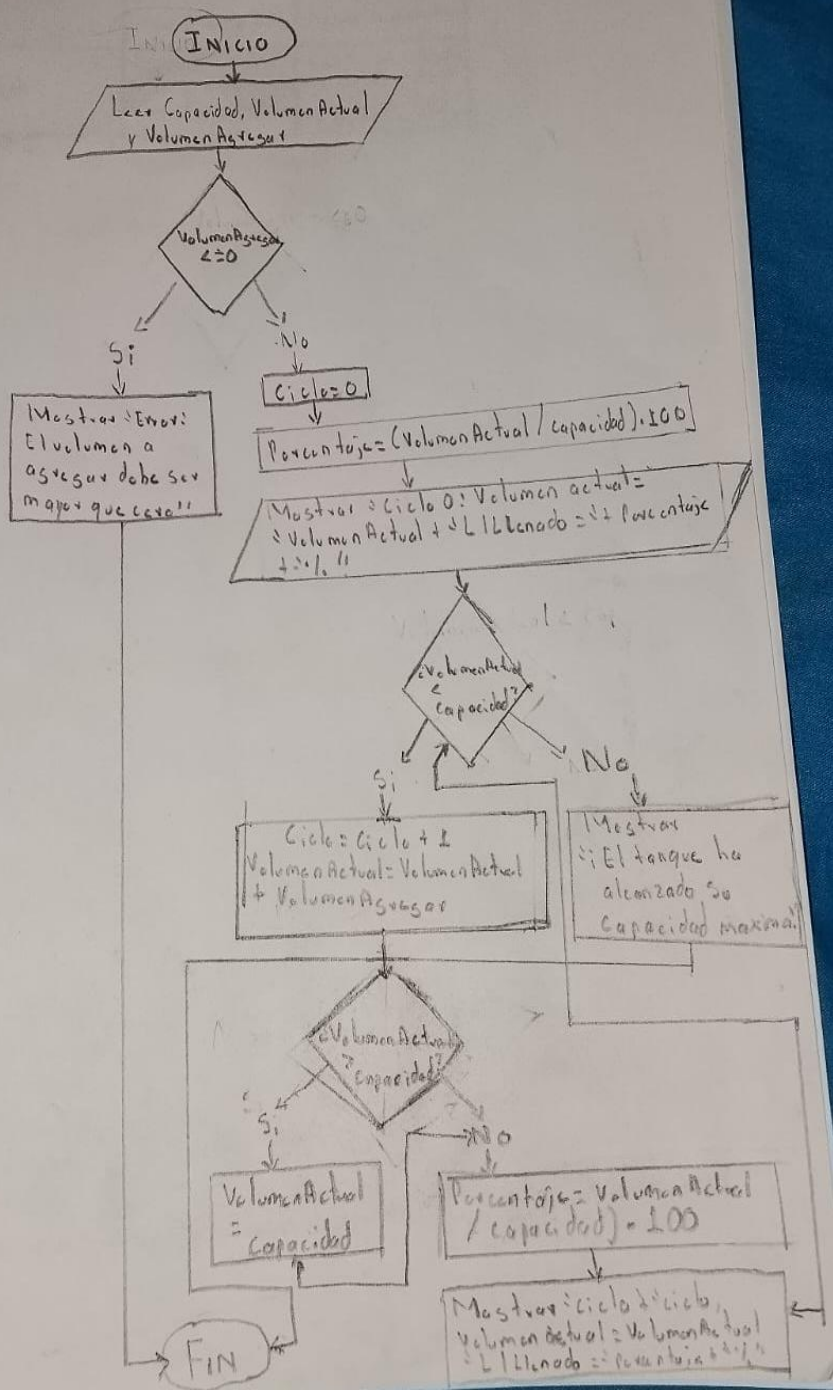
Mientras Volumen Actual < Capacidad hacer
  Ciclo = Ciclo + 1
  Volumen Actual = Volumen Actual + Volumen Agregar

Si Volumen Actual > Capacidad Entonces
  Volumen Actual = Capacidad
Fin Si

Porcentaje = (Volumen Actual / Capacidad) * 100
Mostrar "Ciclo " + Ciclo + ": Volumen actual = " + Volumen Actual + " L | Llenado = " + porcentaje + "% "
Fin Mientras
Mostrar "¡ El tanque ha alcanzado su capacidad máxima! "
Fin Si
Fin
  
```

Diagrama de flujo

- 1) Ent Idar Cor Res Proc Soli Med mod Auu Cal -Ei -s -N -S R -N -P -cu -S -Id -P -Es 2) In v. M L . P l . P S e S c

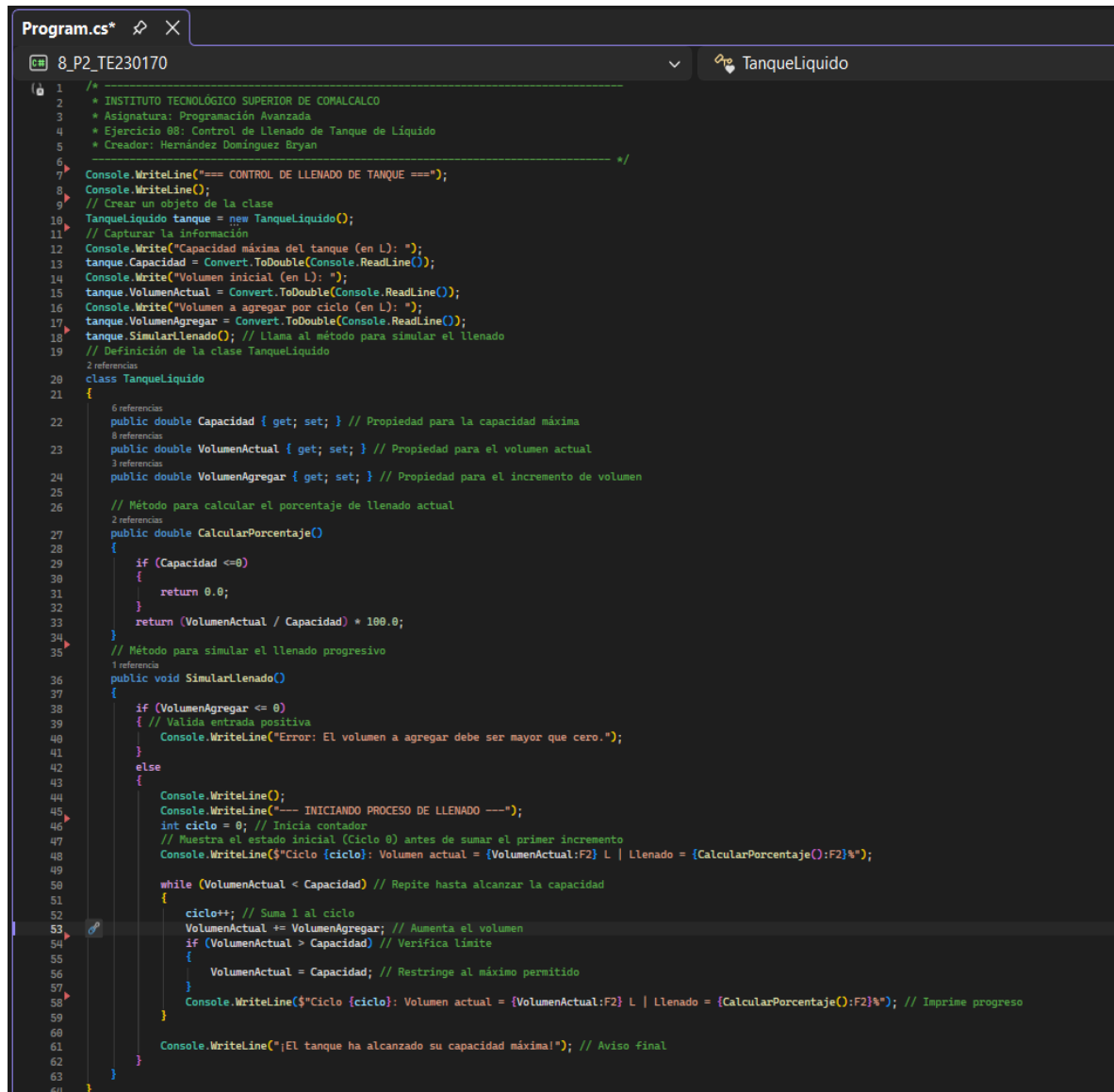


4) Determinar y dibujar una tabla con:

Las clases	Objetos	Propiedades (especificar tipo de dato)	Los métodos (especificar sus parámetros)
Tanque Liquido	Tanque	Capacidad (double) Volumen Actual (double) Volumen Agregado (double)	Calcular porcentaje() Siga parametro rotura Simular llenado() Siga parametros de llenado

Parte 2: Desarrollo en Visual Studio:

1) Código en C#.



```
Program.cs* 8_P2_TE230170 TanqueLiquido
1  /*
2   * INSTITUTO TECNOLÓGICO SUPERIOR DE COMALCALCO
3   * Asignatura: Programación Avanzada
4   * Ejercicio 08: Control de Llenado de Tanque de Líquido
5   * Creador: Hernández Domínguez Bryan
6   */
7  Console.WriteLine("=== CONTROL DE LLENADO DE TANQUE ===");
8  Console.WriteLine();
9  // Crear un objeto de la clase
10 TanqueLiquido tanque = new TanqueLiquido();
11 // Capturar la información
12 Console.Write("Capacidad máxima del tanque (en L): ");
13 tanque.Capacidad = Convert.ToDouble(Console.ReadLine());
14 Console.Write("Volumen inicial (en L): ");
15 tanque.VolumenActual = Convert.ToDouble(Console.ReadLine());
16 Console.Write("Volumen a agregar por ciclo (en L): ");
17 tanque.VolumenAgregar = Convert.ToDouble(Console.ReadLine());
18 tanque.SimularLlenado(); // llama al método para simular el llenado
19 // Definición de la clase TanqueLiquido
20 class TanqueLiquido
21 {
22     6 referencias
23     public double Capacidad { get; set; } // Propiedad para la capacidad máxima
24     8 referencias
25     public double VolumenActual { get; set; } // Propiedad para el volumen actual
26     3 referencias
27     public double VolumenAgregar { get; set; } // Propiedad para el incremento de volumen
28
29     // Método para calcular el porcentaje de llenado actual
30     2 referencias
31     public double CalcularPorcentaje()
32     {
33         if (Capacidad <= 0)
34         {
35             return 0.0;
36         }
37         return (VolumenActual / Capacidad) * 100.0;
38     }
39
40     // Método para simular el llenado progresivo
41     1 referencia
42     public void SimularLlenado()
43     {
44         if (VolumenAgregar <= 0)
45         { // Valida entrada positiva
46             Console.WriteLine("Error: El volumen a agregar debe ser mayor que cero.");
47         }
48         else
49         {
50             Console.WriteLine();
51             Console.WriteLine("--- INICIANDO PROCESO DE LLENADO ---");
52             int ciclo = 0; // Inicia contador
53             // Muestra el estado inicial (Ciclo 0) antes de sumar el primer incremento
54             Console.WriteLine($"Ciclo {ciclo}: Volumen actual = {VolumenActual:F2} L | Llenado = {CalcularPorcentaje():F2}%");
55
56             while (VolumenActual < Capacidad) // Repite hasta alcanzar la capacidad
57             {
58                 ciclo++; // Suma 1 al ciclo
59                 VolumenActual += VolumenAgregar; // Aumenta el volumen
60                 if (VolumenActual > Capacidad) // Verifica límite
61                 {
62                     VolumenActual = Capacidad; // Restringe al máximo permitido
63                 }
64                 Console.WriteLine($"Ciclo {ciclo}: Volumen actual = {VolumenActual:F2} L | Llenado = {CalcularPorcentaje():F2}%"); // Imprime progreso
65             }
66
67             Console.WriteLine("El tanque ha alcanzado su capacidad máxima!"); // Aviso final
68         }
69     }
70 }
```

Fig. 29: Captura del Código del ejercicio 8.

2) El encabezado en la consola.

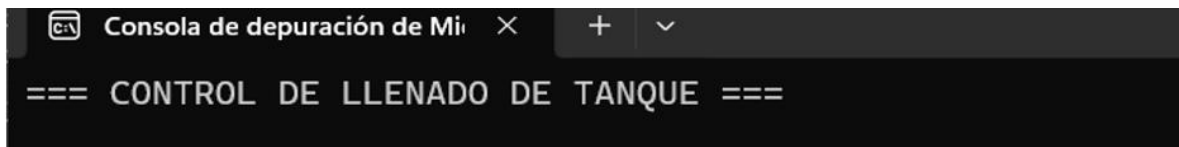


Fig. 30: Captura de la consola mostrando el inicio del ejercicio 8.

3) Evidencia de ejecución con dos Evidencias.

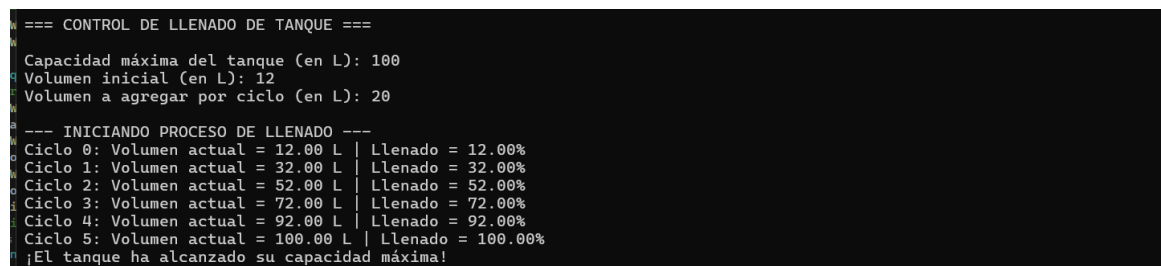


Fig. 31: Ejecución del código 8 (prueba 1).

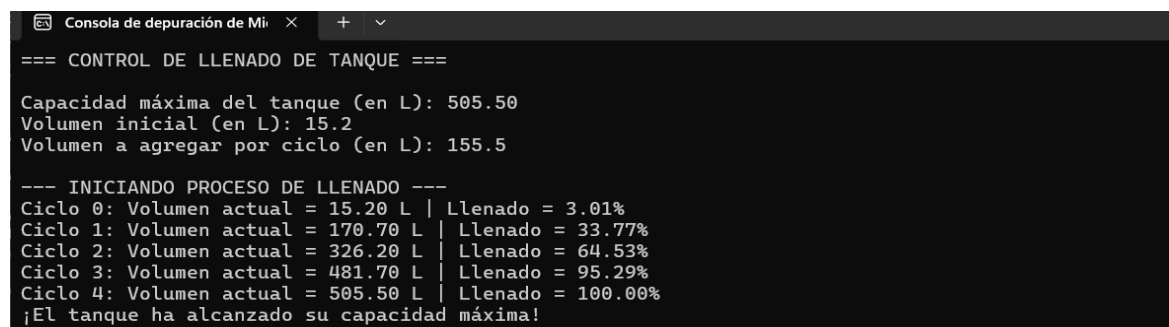


Fig. 32: Ejecución del código 8 (prueba 2).

4) Conclusión.

El uso de estructuras repetitivas (while) e iterativas integradas en una clase en C# permite simular y controlar el proceso interactivo de llenado progresivo de un tanque, garantizando que el sistema no exceda su capacidad máxima configurada.

Ejercicio 9:

Realizado por: Hernández Domínguez Bryan.

Se evaluarán dos bombas eléctricas. Para cada una se introducirá identificador, voltaje, corriente y caudal en litros por minuto. El programa calculará la potencia $P = V \cdot I$ y un indicador didáctico de desempeño igual a caudal/potencia. Después comparará los dos indicadores para señalar cuál bomba entrega más caudal por watt o si existe empate.

Parte 1: Desarrollo a mano:

EJERCICIO 9

1) Entradas

- Bomba 1
- Identificador
- Bomba 1 Voltaje
- Bomba 1 Corriente
- Bomba 1 Caudal
- Bomba 2
- Identificador
- Bomba 2 Voltaje
- Bomba 2 Corriente
- Bomba 2 Caudal

Proceso

Calcular

- Ingresar los valores de bomba 1 (Id, Voltaje, Corriente, Caudal)
- Ingresar los valores de bomba 2 (Id, Voltaje, Corriente, Caudal)
- Calcular la potencia de bomba 1 ($P_1 = \text{Voltaje}_1 \times \text{Corriente}_1$)
- Calcular el desempeño de bomba 1 (Caudal_1 / P_1 , si $P_1 \leq 0$ retorna 0.0W)
- Calcular la potencia de bomba 2 ($P_2 = \text{Voltaje}_2 \times \text{Corriente}_2$)
- Calcular el desempeño de bomba 2 (Caudal_2 / P_2 , si $P_2 \leq 0$ retorna 0.0W)
- Imprimir los resultados de potencia (W) y desempeño (L/min/W) de ambas bombas.
- Evaluar y comparar ambos resultados para emitir una conclusión

Salidas

- Mostrar los resultados de ambas bombas (Potencia y desempeño).
- Mostrar mensaje con la conclusión de rendimiento.

2) Pseudocódigo

Inicio

Leer bomba1.Id
Leer bomba1.Voltaje
Leer bomba1.Corriente
Leer bomba1.Caudal

Leer bomba2.Id
Leer bomba2.Voltaje
Leer bomba2.Corriente
Leer bomba2.Caudal

Potencia₁ = Voltaje₁ x Corriente₁
Si Potencia₁ <= 0 Entonces
Desempeño₁ = 0.0W
Sino
Desempeño₁ = Caudal₁ / Potencia₁
Fin Si

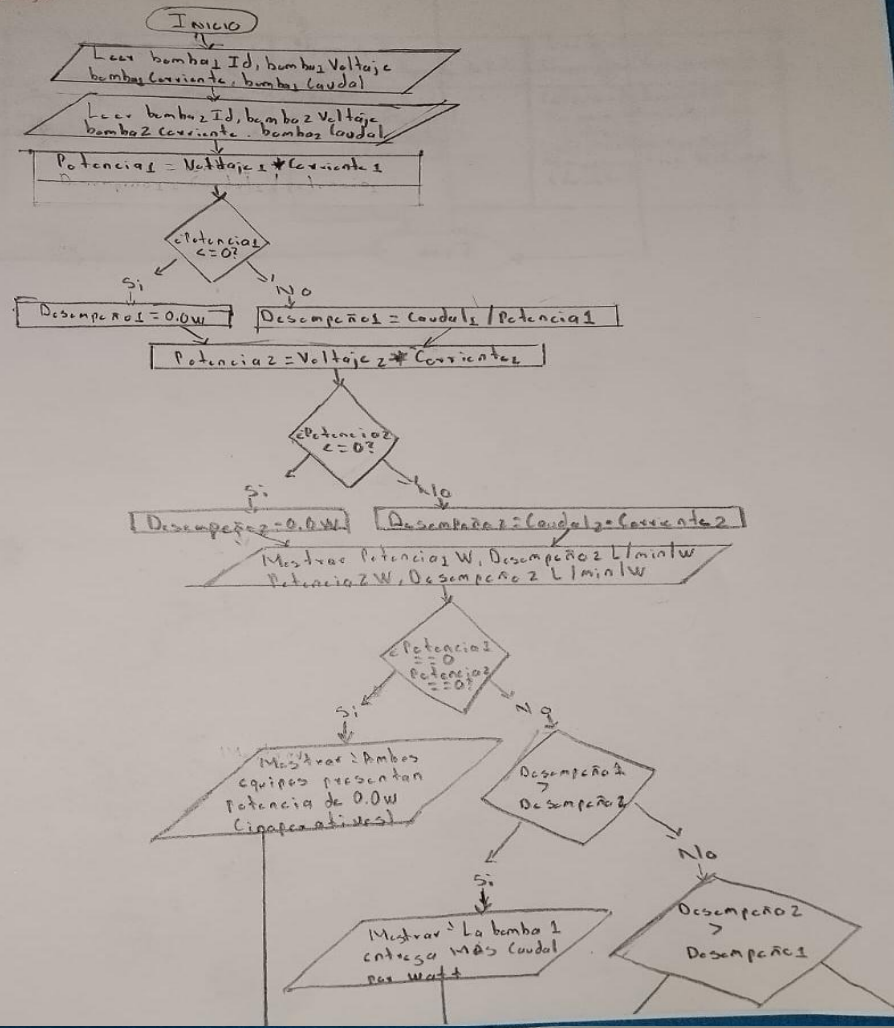
Potencia₂ = Voltaje₂ x Corriente₂
Si Potencia₂ <= 0 Entonces
Desempeño₂ = 0.0W
Sino
Desempeño₂ = Caudal₂ / Potencia₂
Fin Si

Mostrar "Bomba " + Id₁ + ": Potencia₁ = " + Potencia₁ + "W"
Desempeño₁ = " + Desempeño₁ + " L/min/W"

Mostrar "Bomba " + Id₂ + ": Potencia₂ = " + Potencia₂ + "W"
Desempeño₂ = " + Desempeño₂ + " L/min/W"

Si Potencia₁ == 0 y Potencia₂ == 0 Entonces
Mostrar "Conclusión: Ambos equipos presentan potencia de 0.0W (no operativos)"
Sino
Si Desempeño₁ > Desempeño₂ Entonces
Mostrar "Conclusión: La bomba " + Id₁ + " entrega MÁS Caudal por watt."
Sino
Si Desempeño₂ > Desempeño₁ Entonces
Mostrar "Conclusión: La bomba " + Id₂ + " entrega MÁS Caudal por watt."
Sino
Mostrar "Conclusión: Ambos equipos presentan el mismo desempeño."
Fin Si
Fin Si
Fin Si
Fin

3) Diagrama de flujo

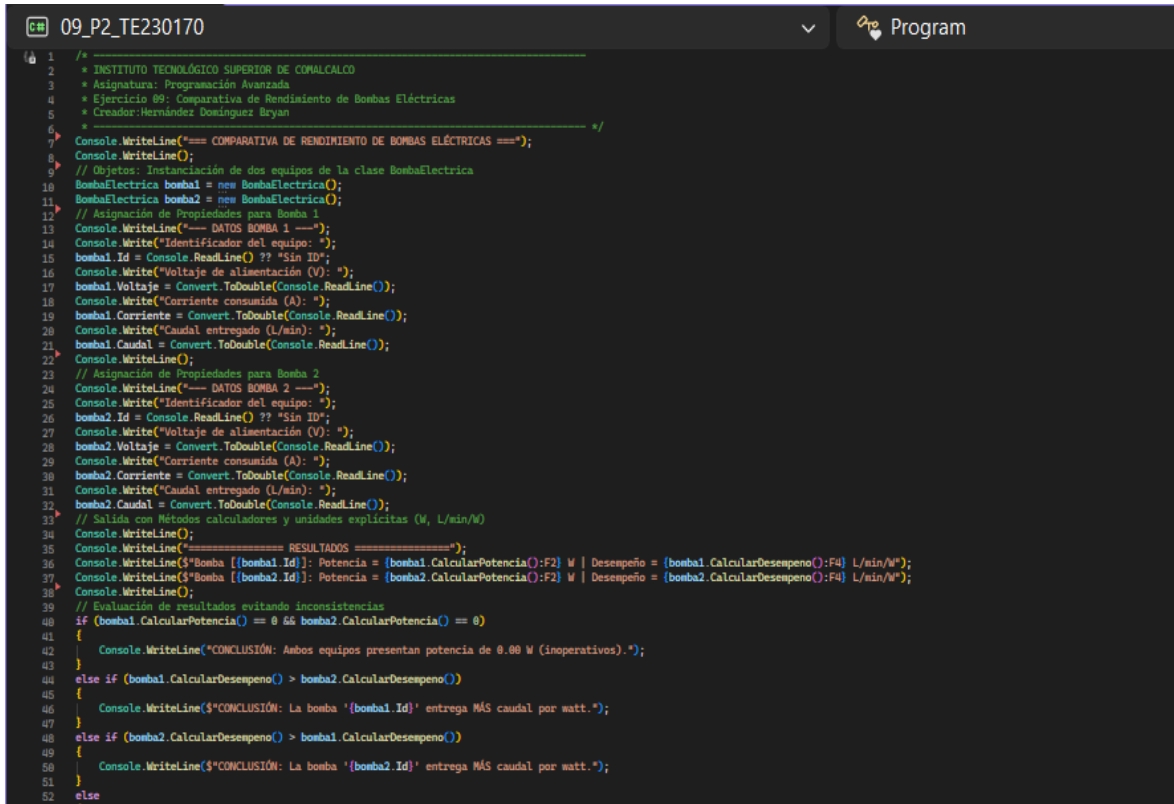


4) Determinar y elaborar una tabla con:

Las clases	Objetos	Propiedades (especificar tipo de dato)	Los métodos (especificar los parámetros)
Bomba Eléctrica	bomba 1 bomba 2	Id (String) Voltaje (double) Corriente (double) Caudal (double)	Calcular Potencia () Sin parámetros Retorna (double) Calcular Desempeño () Sin parámetros Retorna (double)

Parte 2: Desarrollo en Visual Studio:

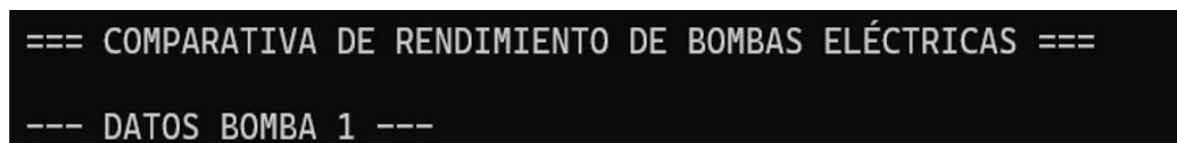
1) Código en C#.



```
1  /*
2  * INSTITUTO TECNOLÓGICO SUPERIOR DE COMALCALCO
3  * Asignatura: Programación Avanzada
4  * Ejercicio 09: Comparativa de Rendimiento de Bombas Eléctricas
5  * Creador: Hernández Domínguez Bryan
6  */
7  Console.WriteLine("=== COMPARATIVA DE RENDIMIENTO DE BOMBAS ELÉCTRICAS ===");
8  Console.WriteLine();
9  // Objetos: Instanciación de dos equipos de la clase BombaElectrica
10 BombaElectrica bomba1 = new BombaElectrica();
11 BombaElectrica bomba2 = new BombaElectrica();
12 // Asignación de Propiedades para Bomba 1
13 Console.WriteLine("--- DATOS BOMBA 1 ---");
14 Console.WriteLine("Identificador del equipo: ");
15 bomba1.Id = Console.ReadLine() ?? "Sin ID";
16 Console.WriteLine("Voltaje de alimentación (V): ");
17 bomba1.Voltaje = Convert.ToDouble(Console.ReadLine());
18 Console.WriteLine("Corriente consumida (A): ");
19 bomba1.Corriente = Convert.ToDouble(Console.ReadLine());
20 Console.WriteLine("Caudal entregado (L/min): ");
21 bomba1.Caudal = Convert.ToDouble(Console.ReadLine());
22 Console.WriteLine();
23 // Asignación de Propiedades para Bomba 2
24 Console.WriteLine("--- DATOS BOMBA 2 ---");
25 Console.WriteLine("Identificador del equipo: ");
26 bomba2.Id = Console.ReadLine() ?? "Sin ID";
27 Console.WriteLine("Voltaje de alimentación (V): ");
28 bomba2.Voltaje = Convert.ToDouble(Console.ReadLine());
29 Console.WriteLine("Corriente consumida (A): ");
30 bomba2.Corriente = Convert.ToDouble(Console.ReadLine());
31 Console.WriteLine("Caudal entregado (L/min): ");
32 bomba2.Caudal = Convert.ToDouble(Console.ReadLine());
33 // Salida con Métodos calculadores y unidades explícitas (W, L/min/W)
34 Console.WriteLine();
35 Console.WriteLine("===== RESULTADOS =====");
36 Console.WriteLine($"Bomba [{bomba1.Id}]: Potencia = {bomba1.CalcularPotencia():F2} W | Desempeño = {bomba1.CalcularDesempeno():F4} L/min/W");
37 Console.WriteLine($"Bomba [{bomba2.Id}]: Potencia = {bomba2.CalcularPotencia():F2} W | Desempeño = {bomba2.CalcularDesempeno():F4} L/min/W");
38 Console.WriteLine();
39 // Evaluación de resultados evitando inconsistencias
40 if (bomba1.CalcularPotencia() == 0 && bomba2.CalcularPotencia() == 0)
41 {
42     Console.WriteLine("CONCLUSIÓN: Ambos equipos presentan potencia de 0.00 W (inoperativos).");
43 }
44 else if (bomba1.CalcularDesempeno() > bomba2.CalcularDesempeno())
45 {
46     Console.WriteLine($"CONCLUSIÓN: La bomba '{bomba1.Id}' entrega MÁS caudal por watt.");
47 }
48 else if (bomba2.CalcularDesempeno() > bomba1.CalcularDesempeno())
49 {
50     Console.WriteLine($"CONCLUSIÓN: La bomba '{bomba2.Id}' entrega MÁS caudal por watt.");
51 }
52 else
```

Fig. 33: Captura del Código del ejercicio 9.

2) El encabezado en la consola.



```
=== COMPARATIVA DE RENDIMIENTO DE BOMBAS ELÉCTRICAS ===
--- DATOS BOMBA 1 ---
```

Fig. 34: Captura de la consola mostrando el inicio del ejercicio 9.

3) Evidencia de ejecución con dos Evidencias.

```
=== COMPARATIVA DE RENDIMIENTO DE BOMBAS ELÉCTRICAS ===

--- DATOS BOMBA 1 ---
Identificador del equipo: Succion
Voltaje de alimentación (V): 220
Corriente consumida (A): 24
Caudal entregado (L/min): 156

--- DATOS BOMBA 2 ---
Identificador del equipo: Riego
Voltaje de alimentación (V): 110
Corriente consumida (A): 5
Caudal entregado (L/min): 154.52

===== RESULTADOS =====
Bomba [Succion]: Potencia = 5280.00 W | Desempeño = 0.0295 L/min/W
Bomba [Riego]: Potencia = 550.00 W | Desempeño = 0.2809 L/min/W

CONCLUSIÓN: La bomba 'Riego' entrega MÁS caudal por watt.
```

Fig. 35: Ejecución del código 9 (prueba 1).

```
=== COMPARATIVA DE RENDIMIENTO DE BOMBAS ELÉCTRICAS ===

--- DATOS BOMBA 1 ---
Identificador del equipo: ABSORCION
Voltaje de alimentación (V): 220
Corriente consumida (A): 0
Caudal entregado (L/min): 156

--- DATOS BOMBA 2 ---
Identificador del equipo: 110
Voltaje de alimentación (V): 110
Corriente consumida (A): 0
Caudal entregado (L/min): 124

===== RESULTADOS =====
Bomba [ABSORCION]: Potencia = 0.00 W | Desempeño = 0.0000 L/min/W
Bomba [110]: Potencia = 0.00 W | Desempeño = 0.0000 L/min/W

CONCLUSIÓN: Ambos equipos presentan potencia de 0.00 W (inoperativos).
```

Fig. 36: Ejecución del código 9 (prueba 2).

4) Conclusión.

La implementación de la Programación Orientada a Objetos en C# permite estructurar adecuadamente las propiedades de los equipos y evaluar múltiples condiciones operativas mediante métodos, automatizando la clasificación del estado de funcionamiento en tiempo real.

Ejercicio 10:

Realizado por: Hernández Domínguez Bryan.

Un técnico desea diagnosticar una cantidad desconocida de motores. Para cada motor se introducirá su identificador y cuatro mediciones de corriente. Las mediciones se capturarán con un ciclo for y se calculará el promedio. Para este ejercicio, un promedio menor o igual que 5 A se considera NORMAL y uno mayor indica REQUIERE MANTENIMIENTO. Después de cada diagnóstico se preguntará si se desea registrar otro motor mediante un ciclo while.

Parte 1: Desarrollo a mano:

EJERCICIO 10

1)

Entradas

Identificador del Motor
Corrientes
Respuesta

Proceso

- Solicitar el identificador del motor
- Mediante un ciclo solicitar las mediciones de corrientes
- Acumular las suma
- Sumas $Suma + Corriente$
- Calcular el promedio Consumido
- $Promedio = Suma / 4$
- Evaluar el promedio
- Si promedio ≤ 5 , O.K. hacer
- Mostrar estado Normal
- Si no mostrar estado
- Requiere Mantenimiento
- Mostrar los resultados
- Preguntar al usuario si desea evaluar otro motor

Salidas

Identificador del Motor
Promedio de las corrientes
Estado de diagnóstico

2) Pseudocódigo

Inicio

respuesta = "S"

Mientras respuesta = "S" hacer

Leer identificador

suma = 0

Para contador = 1 hasta 4 hacer

Leer corriente

suma = suma + corriente

Fin Para

Promedio = suma / 4

Si promedio ≤ 5 Entonces

estado = NORMAL

Si No

estado = "Requiere Mantenimiento"

Fin Si

Mostrar identificador

Mostrar promedio

Mostrar estado

Leer respuesta

Fin Mientras

~~Mostrar~~ ¿C Desea registrar otro motor?

(S/N)

Leer respuesta

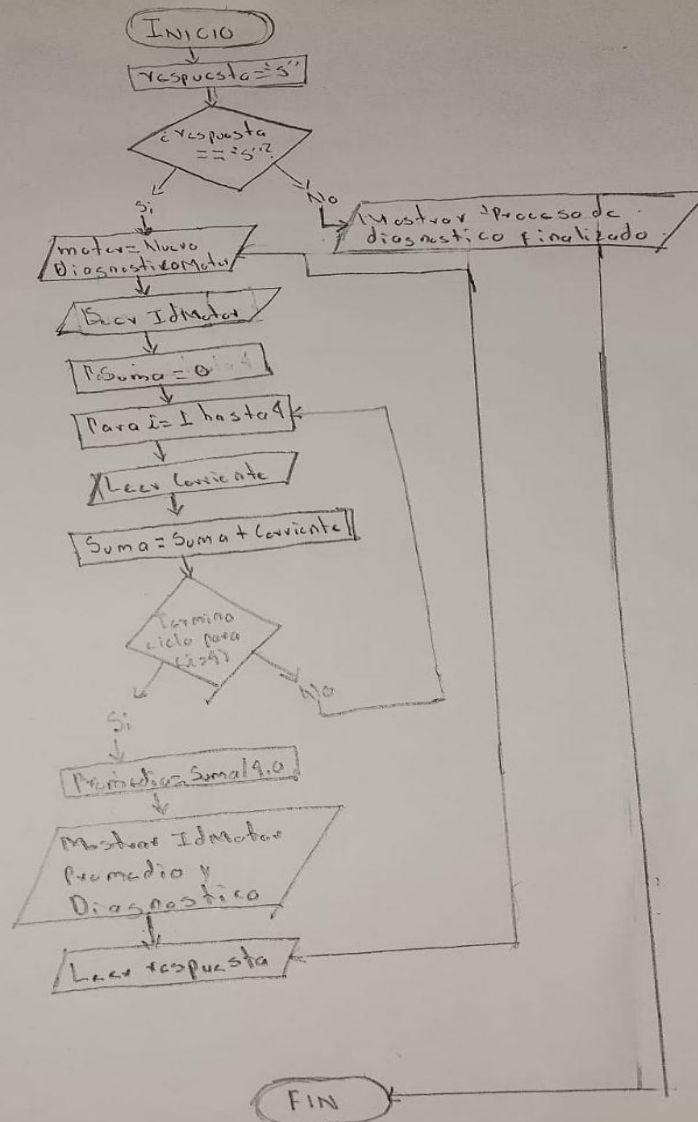
Fin Mientras

Mostrar "Proceso de diagnóstico"

Finalizado exitosamente

FIN.

3) Diagrama de flujo



4) Determinar y elaborar una tabla con

Las clases	Objetos	Propiedades (especificar tipo de datos)	Los métodos (especificar los parámetros)
DiagnosticoMotor	Motor	- IdMotor (string) - Promedio (double)	- ObtenerDiagnostico() - ProcesarDiagnostico()

Parte 2: Desarrollo en Visual Studio:

1) Código en C#.

```
10_P2_TE230170 Program
1  /*
2   * INSTITUTO TECNOLÓGICO SUPERIOR DE COMALCALCO
3   * Asignatura: Programación Avanzada
4   * Ejercicio 10: Diagnóstico Múltiple de Motores Eléctricos
5   * Creador: Hernández Domínguez Bryan
6   */
7  Console.WriteLine("=== DIAGNÓSTICO MÚLTIPLE DE MOTORES ===");
8  Console.WriteLine();
9  // Variable de control inicializada para permitir la primera iteración del ciclo while
10 string respuesta = "s";
11 while (respuesta == "s")
12 {
13     // Crear un objeto de la clase DiagnosticoMotor
14     DiagnosticoMotor motor = new DiagnosticoMotor();
15     // Capturar el identificador del motor
16     Console.Write("Ingrese el identificador del motor: ");
17     motor.IdMotor = Console.ReadLine() ?? "Sin ID";
18     double sumaCorrientes = 0;
19     // Captura de las 4 mediciones mediante un ciclo for
20     for (int i = 1; i <= 4; i++)
21     {
22         Console.Write($"Ingrese la medición de corriente #{i} (A): ");
23         double corriente = Convert.ToDouble(Console.ReadLine());
24         sumaCorrientes = sumaCorrientes + corriente;
25     }
26     // Llama al método para procesar la información y calcular el promedio
27     motor.ProcesarDiagnostico(sumaCorrientes);
28     // Lectura de control al final del ciclo para preguntar si desea registrar otro motor
29     Console.WriteLine();
30     Console.Write("¿Desea registrar otro motor? (s/n): ");
31     respuesta = Console.ReadLine() ?? "n";
32     Console.WriteLine();
33 }
34 Console.WriteLine("Proceso de diagnóstico finalizado exitosamente.");
35 // -----
36 // DEFINICIÓN DE LA CLASE
37 // -----
38 2 referencias
39 class DiagnosticoMotor
40 {
41     2 referencias
42     public string IdMotor { get; set; } = ""; // Propiedad para el identificador
43     3 referencias
44     public double Promedio { get; set; } // Propiedad para el promedio de corriente
45     // Método para determinar la clasificación según el promedio
46     1 referencia
47     public string ObtenerDiagnostico()
48     {
49         if (Promedio <= 5.0)
50         {
51             return "NORMAL";
52         }
53         else
54         {
55             return "REQUIERE MANTENIMIENTO";
56         }
57     }
58     // Método para procesar los datos del motor
59     1 referencia
60     public void ProcesarDiagnostico(double sumaCorrientes)
61     {
62         Promedio = sumaCorrientes / 4.0; // Cálculo del promedio (4.0 evita división entera)
63         Console.WriteLine();
64         Console.WriteLine($"--- RESULTADOS DEL MOTOR: {IdMotor} ---");
65         Console.WriteLine($"Corriente Promedio: {Promedio:F2} A");
66         Console.WriteLine($"Diagnóstico Final: {ObtenerDiagnostico()}");
67     }
68 }
```

Fig. 37: Captura del Código del ejercicio 10.

2) El encabezado en la consola.

```
=== DIAGNÓSTICO MÚLTIPLE DE MOTORES ===  
Ingrese el identificador del motor: Fabrica
```

Fig. 38: Captura de la consola mostrando el inicio del ejercicio 10.

3) Evidencia de ejecución con dos Evidencias.

```
=== DIAGNÓSTICO MÚLTIPLE DE MOTORES ===  
Ingrese el identificador del motor: Fabrica  
Ingrese la medición de corriente #1 (A): 4  
Ingrese la medición de corriente #2 (A): 3  
Ingrese la medición de corriente #3 (A): 2  
Ingrese la medición de corriente #4 (A): 1  
  
--- RESULTADOS DEL MOTOR: Fabrica ---  
Corriente Promedio: 2.50 A  
Diagnóstico Final: NORMAL  
  
¿Desea registrar otro motor? (s/n): s  
  
Ingrese el identificador del motor: Pozo  
Ingrese la medición de corriente #1 (A): 10  
Ingrese la medición de corriente #2 (A): 14  
Ingrese la medición de corriente #3 (A): 12.5  
Ingrese la medición de corriente #4 (A): 13.5  
  
--- RESULTADOS DEL MOTOR: Pozo ---  
Corriente Promedio: 12.50 A  
Diagnóstico Final: REQUIERE MANTENIMIENTO  
  
¿Desea registrar otro motor? (s/n): n  
  
Proceso de diagnóstico finalizado exitosamente.
```

Fig. 39: Ejecución del código 10 (prueba 1).

```
=== DIAGNÓSTICO MÚLTIPLE DE MOTORES ===  
  
Ingrese el identificador del motor: Postal  
Ingrese la medición de corriente #1 (A): 2.5  
Ingrese la medición de corriente #2 (A): 5.5  
Ingrese la medición de corriente #3 (A): 1.3  
Ingrese la medición de corriente #4 (A): 4.1  
  
--- RESULTADOS DEL MOTOR: Postal ---  
Corriente Promedio: 3.35 A  
Diagnóstico Final: NORMAL  
  
¿Desea registrar otro motor? (s/n): S  
  
Proceso de diagnóstico finalizado exitosamente.
```

Fig. 40: Ejecución del código 10 (prueba 2).

4) Conclusión.

La combinación de ciclos while y for junto con la Programación Orientada a Objetos en C# permite procesar lecturas periódicas de corriente para diagnosticar múltiples motores de forma continua y determinar si requieren mantenimiento de manera precisa.

CONCLUSIÓN GENERAL.

El desarrollo de este problemario consolidó la aplicación de la Programación Orientada a Objetos en C# orientada a la ingeniería. Mediante el encapsulamiento de datos en clases, la integración de métodos modulares y el uso de estructuras iterativas y condicionales (for, while, if/else), se modelaron y controlaron sistemas físicos reales (motores, cilindros, temperaturas, baterías, servomotores y celdas automatizadas). Este enfoque demostró cómo la abstracción de software permite construir soluciones robustas, escalables y con validaciones de seguridad eficientes para procesos industriales y mecatrónicos.